

Intro2SE - Design - 6

SmartATS

Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS



Table of Contents

1	Member Contribution Assessment	5
2	Conceptual Model.....	8
3	Architectural Design.....	10
3.1	Architecture Diagram.....	10
3.1.1	Ingestion Flow Architecture Diagram	10
3.1.2	Scheduling Flow Architecture Diagram	11
3.1.3	Cross-Cutting: ABAC Authorization Architecture.....	12
3.1.4	AI Orchestration.....	13
3.2	Class Diagram.....	14
3.2.1	Ingestion Subsystem Class Diagram.....	14
3.2.2	Data Crawling Subsystem Class Diagram	17
3.2.3	AI Orchestration.....	18
3.2.4	Authorization & ABAC Subsystem Class Diagram	19
3.2.5	Scheduling Subsystem Class Diagram.....	21
3.3	Class Specifications.....	24
3.3.1	Class Workspace Provider	24
3.3.2	Class IngestionGateway.....	25
3.3.3	Class Resume Pipeline.....	27
3.3.4	Class Parser Service.....	28
3.3.5	GitHubScraper Class Specification.....	29
3.3.6	LinkedIn Scraper Service Class	31
3.3.7	Supporting Domain Model Classes	32

3.3.8	AI Orchestration Class.....	33
3.3.9	Scheduling Service Class.....	34
3.3.10	Google Calendar Service Class.....	35
3.3.11	Class CalendarEventService	36
3.3.12	Class EmailNotifier.....	37
3.3.13	Class ABACEngine	38
3.3.14	Class AuthDependencies	38
4	Data Design	40
4.1	Data Diagram.....	40
4.1.1	Asynchronous Crawling Architecture – Enrichment Data Flow	40
4.1.2	Real-time Document Ingestion and Verification Data Flow	43
4.1.3	Interview Scheduling Data Flow	44
4.1.4	ABAC Masking Data Flow.....	45
4.2	Data Specification.....	47
4.2.1.	Idempotent Merge Strategy for Enriched Candidate Profiles.....	47
4.2.2.	Scheduling Data Specification.....	50
4.2.3.	ABAC Data Specification.....	52
5	User Interface and User Experience Design	54
5.1.	Screen Diagram.....	54
5.1.1.	Global Screen Navigation Matrix	54
5.1.2.	AI Orchestration.....	55
5.1.3.	Enriched Candidate Profile View: Component Hierarchy.....	56
5.2.	Screen Specifications.....	57
5.2.1.	Screen 1 — Ingestion & Workspace View	57
5.2.2.	Screen 2 - Specification and Event Handling	59

5.2.3.	Screen 3 - Semantic Ranking Results	60
5.2.4.	Screen 4 - Chat AI Assistant View	61
5.2.5.	Screen 5 - Enriched Candidate Profile View State management:.....	62
5.2.6.	Screen 6 – Interview Scheduling View.....	64
6	AI Usage Declaration.....	67
7	Presentation.....	69
8	Reflective Report	70

Software Design

1 Member Contribution Assessment

24127254 - Hồ Đình Trí - 25%

- **Task Evidence:**
 - **Task Details:**
 - **AP-47: Design Ingestion Gateway Infrastructure & WebSocket Router Backend:** Designed the private Azure Blob Storage container structure, message event contract formats for Azure Service Bus, and persistent connection mechanisms for the WebSocket Analysis Hub.
 - **AP-48: Author Class Specifications & Screen Event handling for Ingestion Module:** Modeled class structures for the ingestion pipeline, wrote formal property/method specifications for PDFStreamingService, and documented full UI event handling matrixes for *Screen 1: Ingestion Drop-Zone & Workspace Layout*.
 - **Visual Evidence:**

▼ AP-3	Epic 2: Profile Ingestion Pipeline & Workspace ...		Đinh Trí		Đinh Trí	Medium	DONE ▼	Done
☒ AP-48	Author Class Specifications & Screen Event ...		Đinh Trí		Đinh Trí	Medium	DONE ▼	Done
☒ AP-47	Design Ingestion Gateway Infrastructure & ...		Đinh Trí		Đinh Trí	Medium	DONE ▼	Done

24127382 - Trần Nhật Hoàng - 25%

- **Task Evidence:**
 - **Task Details:**
 - **AP-49: Design Multi-Agent Core Orchestration Loop & pgvector Schema:** Defined Agent-to-Agent interaction protocols via Model Context Protocol (MCP) SDK and designed the 1536-dimensional vector embedding storage layout in PostgreSQL using pgvector.

- **AP-50: Author Class Specifications & Screen Event handling for AI Ranking Module:** Modeled class structures for AI orchestrators, wrote detailed class specifications for AIOrchestrationLoop and SemanticRankingEngine, and authored screen specifications for *Screen 2: Semantic Ranking Results* and *Screen 3: Chat AI Assistant View*.
- **Visual Evidence:**

▼ AP-4	Epic 3: AI-Driven Resume Parsing & Semantic ...	HT	Hoang Tran	Đinh Trí	Medium	DONE ▼	Done
AP-50	Author Class Specifications & Screen Event h...	HT	Hoang Tran	Đinh Trí	Medium	DONE ▼	Done
AP-49	Design Multi-Agent Core Orchestration Loop ...	HT	Hoang Tran	Đinh Trí	Medium	DONE ▼	Done

24127252 - Nguyễn Khánh Toàn - 25%

- **Task Evidence:**
 - **Task Details:**
 - **AP-51: Design Asynchronous Crawling Engines & Idempotent Data Merge Rules:** Architected background consumer workers for GitHub/LinkedIn (Proxycurl) API data extraction and defined idempotent data merge algorithms into PostgreSQL JSONB structures.
 - **AP-52: Author Class Specifications & Screen Event handling for Profile Enrichment Module:** Modeled class structures for enrichment scrapers, wrote specifications for GitHubScraper and LinkedInAdapter, and authored screen specifications for *Screen 4: Enriched Candidate Profile View*.
 - **Visual Evidence:**

▼ AP-5	Epic 4: Cross-Channel Profile Enrichme...	TK	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ▼	Done
AP-52	Author Class Specifications & Screen Event h...	TK	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ▼	Done
AP-51	Design Asynchronous Crawling Engines & Id...	TK	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ▼	Done

24127337 - Nguyễn Tiến Cường - 25%

- **Task Evidence:**
 - **Task Details:**
 - **AP-53: Design Dynamic ABAC Masking Pipeline & Sweep-Line Overlap Scheduler:** Designed recursive JSON payload sanitization middleware for attribute-based access control (ABAC) and structured free/busy array interval algorithms for the Sweep-Line scheduler.

- **AP-54: Author Class Specifications & Screen Event handling for Security & Scheduling Module:** Modeled class structures for security and scheduling services, wrote class specifications for ABACPolicyEngine and SweepLineScheduler, and authored screen specifications for *Screen 5: Masked PII Interviewer View* and *Screen 6: Contextual Interview Scheduler Screen*.

- **Visual Evidence:**

▼ AP-6 Epic 5: Automated Interview Scheduling... ↗ +	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ▼	Done
☒ AP-54 Author Class Specifications & Screen Event h...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ▼	Done
☒ AP-53 Design Dynamic ABAC Masking Pipeline & S...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ▼	Done

Written by: 24127337 - Nguyễn Tiến Cường
 Edited by: 24127254 - Hồ Đình Trí
 Reviewed by: 24127252 - Nguyễn Khánh Toàn



The **SmartATS** database is architected around **Job Postings** and **Candidate Applications**, providing a centralized data model for end-to-end recruitment, AI enrichment, and security auditability:

- **Job Postings & Applications:** A **Job Posting** (*DRAFT* → *PUBLISHED* → *CLOSED* → *ARCHIVED*) defines role specifications, salary bounds, and skill arrays. Candidates submit **Applications** (*SUBMITTED* → *SCREENING* → *INTERVIEW* → *OFFER* → *HIRED* → *REJECTED* → *WITHDRAWN*) linked to specific **Resumes**, capturing preferences, skill ratings (*JSON*), and motivational metrics.
- **Users, Sessions & Interviewers:** System **Users** (*hr, tech_lead, admin, recruiter, interviewer*) are authenticated via **User Sessions** (*JTI tokens*). Specialized **Interviewers** connect external calendar providers (*Google Calendar API*) to manage availability.
- **Candidates & Two-Stage CV Review:** **Candidates** maintain diversity details, education, and resume files. Before interview scheduling, candidates undergo a two-stage **CV Review** process conducted by *HR* and *Tech Lead* reviewers (*pending, approved, rejected*).
- **Scheduling & Confirmed Slots:** Upon review approval, **Confirmed Slots** link candidates with participating interviewer arrays, tracking meeting event IDs and automated notifications (*Email, Slack*).
- **Profile Enrichment & AI Analytics:** The **Enrichment Profile** asynchronously aggregates external footprint data (*GitHub repositories, LinkedIn profiles*) to update skill matrices, generate semantic tags, and compute *AI match confidence scores*.
- **ABAC Security & System Audit Logs:** Sensitive candidate attributes are protected by **ABAC Policies** using dynamic masking rules (*passthrough, redact*).

All system actions, legacy accesses, LLM token usages/costs, and external API rate limits are recorded across dedicated **Audit Logs**.

3 Architectural Design

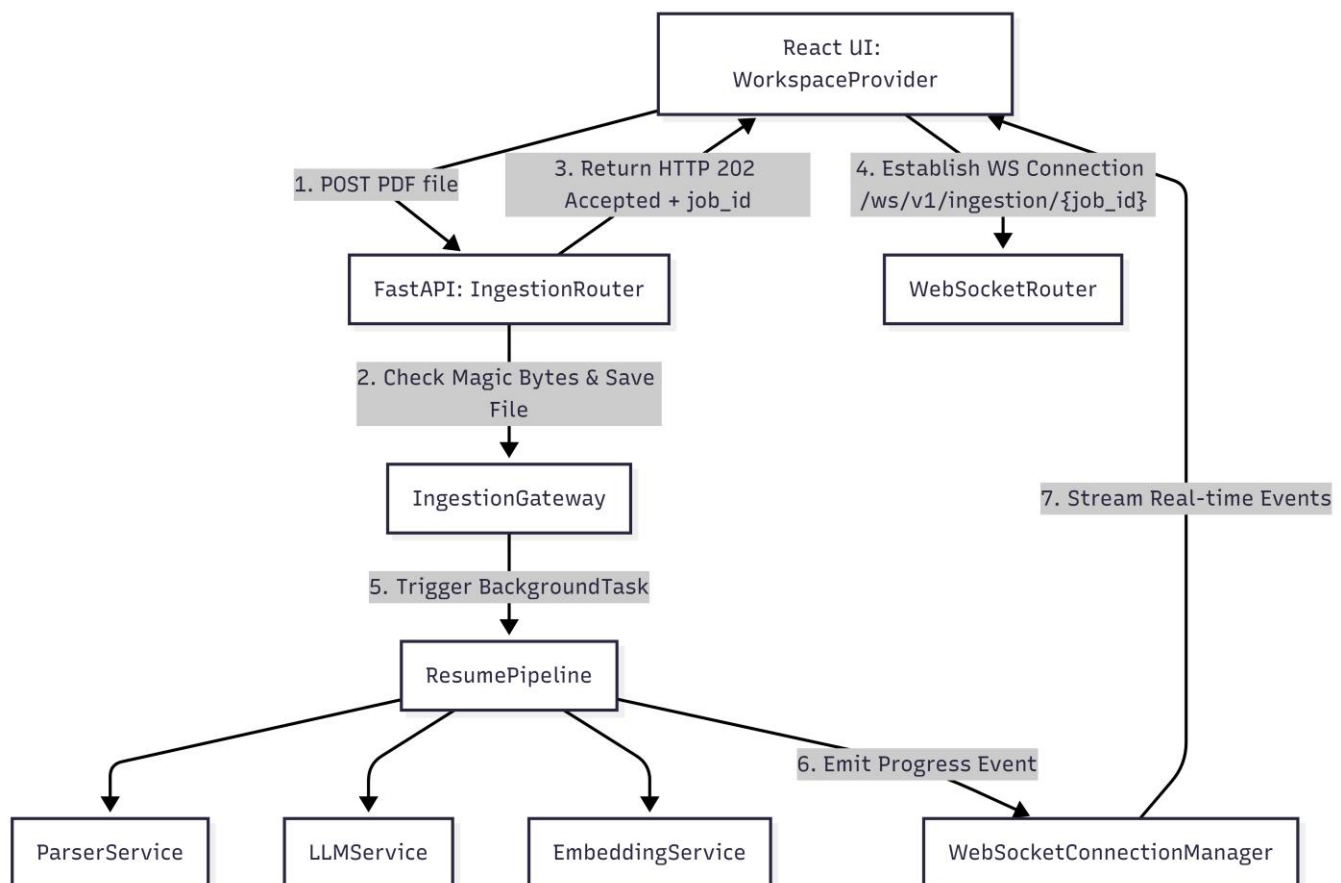
3.1 Architecture Diagram

3.1.1 Ingestion Flow Architecture Diagram

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường



The **Profile Ingestion Module** processes candidate resumes asynchronously through a multi-tier pipeline. It operates through the following structured execution sequence:

- **PDF Payload Upload:** The React UI: WorkspaceProvider initiates the ingestion cycle by sending an asynchronous multipart request with the raw candidate PDF file to the backend API layer.
- **Security Validation & File Persistence:** The FastAPI: IngestionRouter validates binary stream integrity via Magic Bytes checks to prevent malicious uploads before persisting the asset via IngestionGateway.
- **Non-blocking Response & Real-time Handshake:** The backend returns an immediate HTTP 202 Accepted status with a job_id, enabling the UI to establish a duplex WebSocket connection via WebSocketRouter.
- **Pipeline Execution & Real-time Telemetry:** The ResumePipeline triggers background extraction workers (ParserService, LLMService, EmbeddingService) and streams live progress updates back to the UI via WebSocketConnectionManager.

3.1.2 Scheduling Flow Architecture Diagram

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127252 - Nguyễn Khánh Toàn

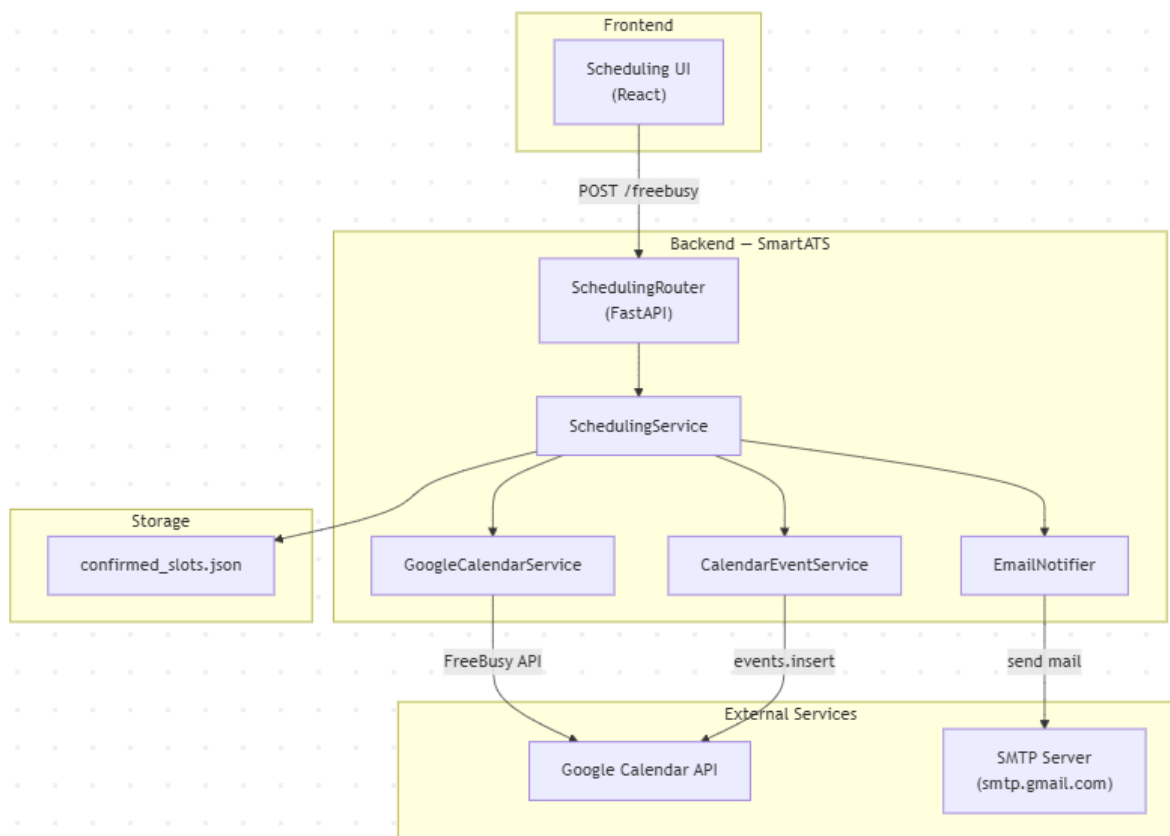
Reviewed by: 24127254 - Hồ Đình Trí

The **Scheduling Module** coordinates the end-to-end interview arrangement workflow among HR recruiters, candidates, and technical interviewers. It operates through the following structured execution sequence:

- **Stakeholder Coordination:** The module serves as a central orchestrator connecting HR recruiters, prospective candidates, and technical interviewers within a unified scheduling pipeline.
- **External Calendar Integration:** It directly integrates with the **Google Calendar API** to query real-time calendar availability of designated interviewers and automatically generate calendar events upon confirmation.
- **Availability Query Delegation:** Built upon a decoupled client-server architecture, the frontend client dispatches availability queries to the FastAPI backend, which

delegates the request to the **Google Calendar FreeBusy API** to fetch free/busy time slots.

- **Event Creation & Participant Notification:** Once a mutually agreed time slot is selected, the module automatically creates a confirmed calendar event and dispatches automated confirmation emails to all participating parties via **SMTP**.
- **Workflow Optimization:** The underlying design streamlines the entire interview scheduling lifecycle, successfully eliminating the need for manual, back-and-forth email coordination.



3.1.3 Cross-Cutting: ABAC Authorization Architecture

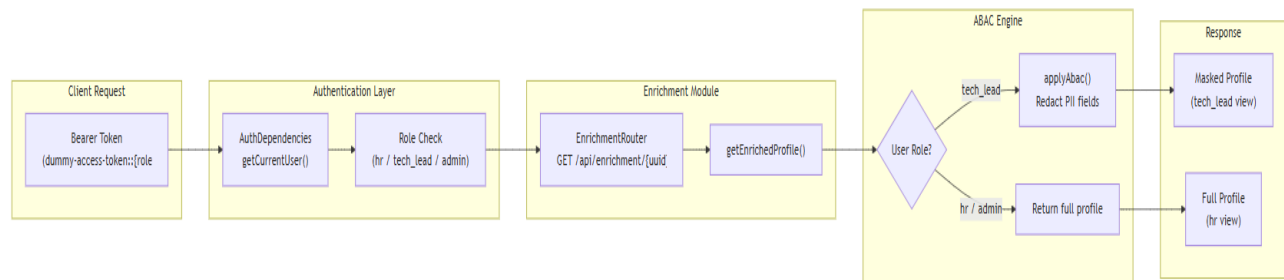
Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

The **ABAC Authorization Module** enforces fine-grained security policies to protect Personally Identifiable Information (PII) based on user roles. It operates through the following structured execution sequence:

- **Role-Based Context Resolution:** Upon receiving a data request, the server extracts and resolves the requesting user's role directly from their authentication token.
- **Dynamic PII Redaction:** If the user holds a restricted role (e.g., Technical Lead), the system recursively redacts sensitive fields such as full name, email, phone number, address, and salary expectation prior to response transmission.
- **Least Privilege Enforcement:** Sensitive attributes are stripped to prevent unauthorized PII exposure, adhering strictly to the principle of least privilege for non-recruitment personnel.
- **Full Data Visibility:** The HR Manager role retains unrestricted data access to facilitate complete recruitment workflow management.

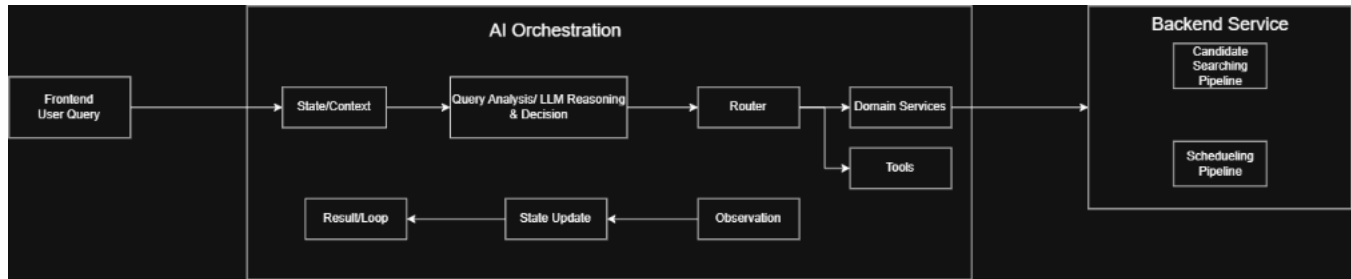


3.1.4 AI Orchestration

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127337 - Nguyễn Tiến Cường



The **AI Orchestration Module** manages dynamic task execution and intelligent decision-making using an LLM-driven feedback loop. It operates through the following structured execution sequence:

- **Query Ingestion & Context Tracking:** The frontend forwards the user query to the AI Orchestration layer, which initializes and maintains the current task state and context.
- **LLM Action Planning:** The core LLM analyzes the request alongside historical context to determine the optimal plan of action and select appropriate execution steps.
- **Dynamic Tool Routing:** Based on the generated decision, the Router dispatches execution commands to specific backend services or pipelines (e.g., Candidate Searching Pipeline or Scheduling Pipeline).
- **Observation Gathering & State Update:** Execution outputs are collected as observations to update the system state, feeding back into the orchestration loop until the task completes and returns the final result to the user.

3.2 Class Diagram

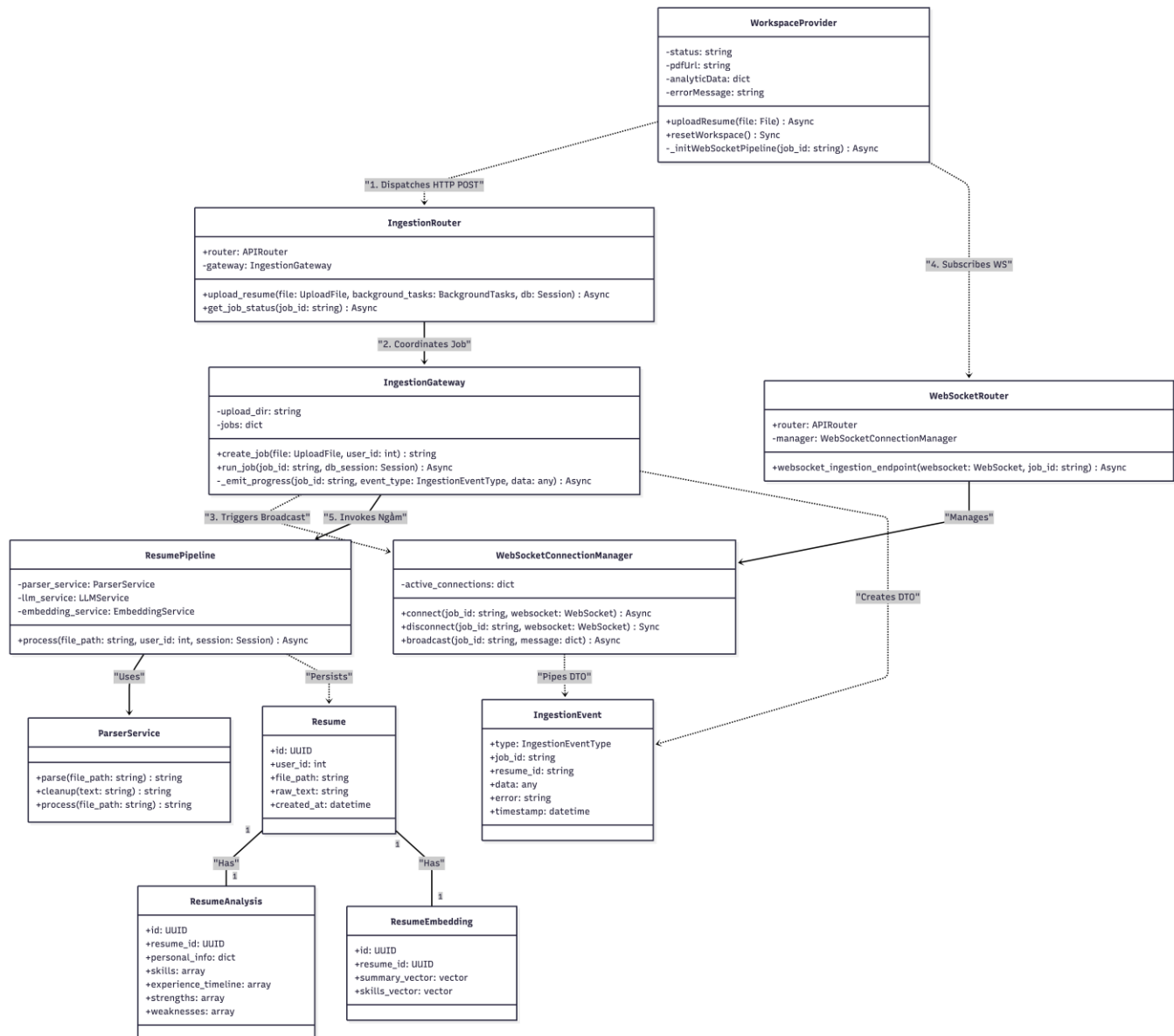
3.2.1 Ingestion Subsystem Class Diagram

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127382 - Trần Nhật Hoàng

The following class diagram documents the structural design of the Ingestion Subsystem implemented for the SmartATS Applicant Tracking System. It captures the essential object classes responsible for receiving raw multi-part file uploads , executing real-time data verification pipelines , managing state transitions , and broadcasting milestone telemetry via persistent WebSocket connections.



Core Service, API and Persistence Classes:

- **WorkspaceProvider:** Acts as the single source of truth for frontend state management. It encapsulates reactivity logic for UI loading indicators, buffers the native PDF binary stream via object URLs, and registers deep event hooks to reflect real-time background processing states.
- **IngestionRouter:** A high-performance FastAPI routing gateway that exposes the primary RESTful endpoint POST `/api/v1/ingestion/resumes` to accept multi-part file payloads. It executes initial file validation constraints, performs binary magic byte inspection to mitigate file-extension spoofing vulnerabilities, and returns non-blocking 202 Accepted tokens.

- **IngestionGateway:** The centralized application boundary service coordinating execution contexts. It creates unique execution tracking UUIDs, commits uploaded binaries to temporary disk storage, provisions user authorization bindings, and handles asynchronous worker initialization.
- **WebSocketRouter:** Exposes the real-time endpoint `/ws/v1/ingestion/{job_id}`. It handles incoming connection upgrades, enforces operational handshake checks, and maintains an active long-poll interface with the client layer.
- **WebSocketConnectionManager:** An infrastructure-level utility component that tracks active socket subscriptions mapped against unique job identifiers. It handles connection pooling, performs routine socket health monitoring, and executes thread-safe event broadcasting.
- **ResumePipeline:** The overarching domain pipeline that sequentializes resume processing workflows. It connects the data ingestion layer with core domain services, passing transactional states dynamically from text parsing to multi-agent LLM analysis and vector embedding generation.
- **ParserService:** A stateless domain service handling layout-aware data extraction from binary text blobs. It parses unstructured PDF assets, normalizes whitespace and encoding structures, and builds sanitized raw text inputs optimized for downstream AI operations.

Domain and Data Model Classes

- **IngestionEvent / IngestionEventType:** Structured data transfer objects (DTOs) defining the operational messaging contract between the client and server. They encapsulate precise state tags including `INGESTION_QUEUED`, `PARSING_STARTED`, `ANALYSIS_STARTED`, and `AI_ANALYTICS_COMPLETE`.
- **Resume:** Represents the persistent domain model entity capturing fundamental resume metadata, including structural file storage paths, raw text extractions, ownership foreign keys (`user_id`), and tracking timestamps.
- **ResumeAnalysis:** Models the rich, structured domain aggregate generated by the LLM core, including normalized personal information records, arrayed skill matrices, chronological experience blocks, and computed profile evaluation scores.

- **ResumeEmbedding:** Represents the persistent vector database entity holding distinct multidimensional float arrays generated by the sentence embedding models for skills, professional histories, and executive profiles.

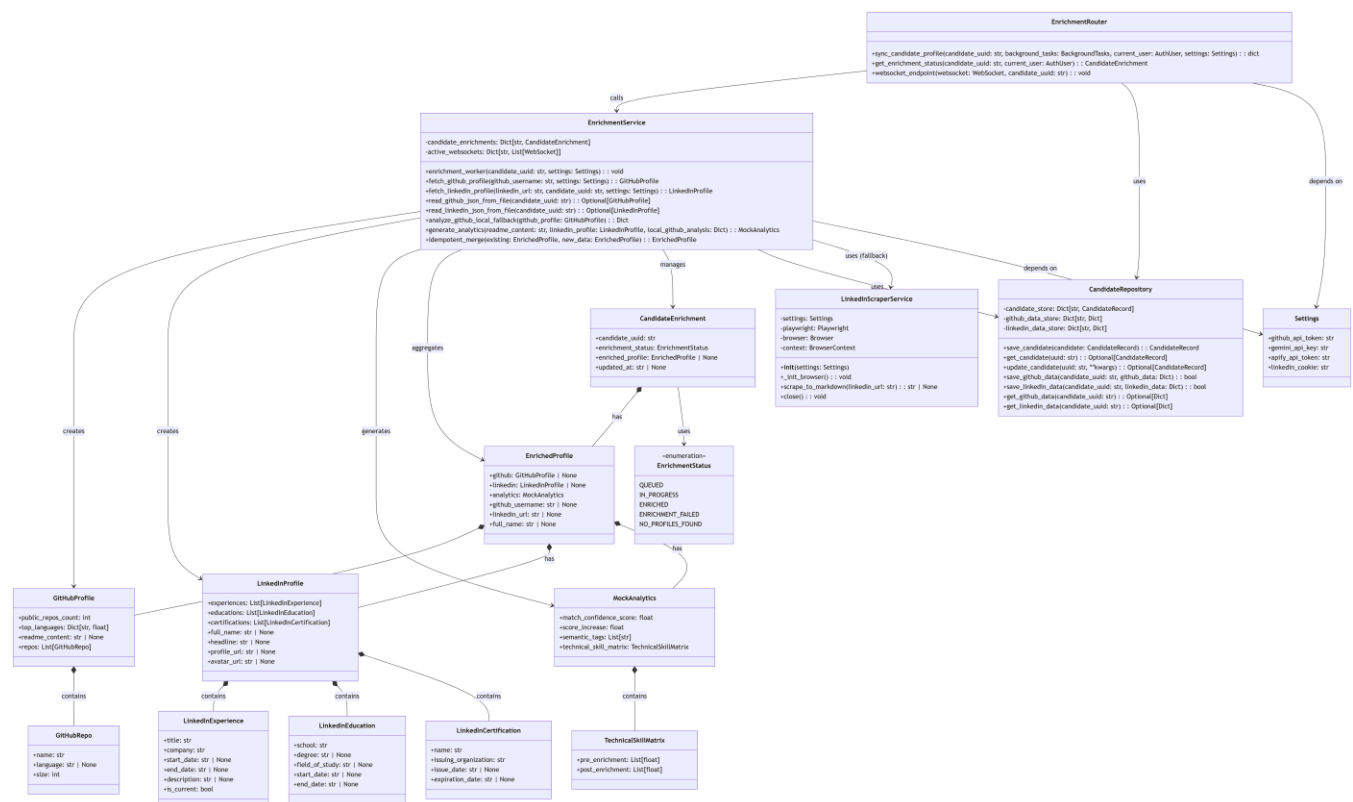
3.2.2 Data Crawling Subsystem Class Diagram

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127254 - Hồ Đình Trí

The following class diagram documents the actual Data Crawling Subsystem implemented for the Profile Enrichment module of the SmartATS Applicant Tracking System. It shows the classes responsible for fetching, processing, and storing candidate profile data collected from GitHub and LinkedIn.



Link picture: <https://drive.google.com/file/d/1ME0SU92AcvfGJBtO7IsgA1mJ-d62vdK3/view?usp=sharing>

Core service and API classes:

- **EnrichmentService** – orchestrates the pipeline: fetches GitHub/LinkedIn data, manages the JSON cache, generates analytics, and merges enriched profiles idempotently.
- **EnrichmentRouter** – exposes the FastAPI endpoints for sync, status check, and real-time WebSocket updates.
- **CandidateRepository** – in-memory persistence gateway storing candidate records and raw GitHub/LinkedIn JSON payloads.
- **LinkedInScraperService** – Playwright-based fallback scraper that converts a LinkedIn profile page into Markdown.
- **Settings** – centralised config holding the GitHub, Gemini, and Apify API tokens plus the LinkedIn cookie.

Domain / data model classes:

- **GitHubProfile & GitHubRepo** – repo count, top languages, README content, and per-repository details.
- **LinkedInProfile, LinkedInExperience, LinkedInEducation, LinkedInCertification** – candidate identity, work history, education, and certifications scraped from LinkedIn.
- **MockAnalytics & TechnicalSkillMatrix** – match-confidence score, score increase, tags, and pre-/post-enrichment skill radar values.
- **EnrichedProfile** – aggregate root combining GitHubProfile, LinkedInProfile, and MockAnalytics for one candidate.
- **CandidateEnrichment & EnrichmentStatus** – track a job's lifecycle through QUEUED → IN_PROGRESS → ENRICHED / ENRICHMENT_FAILED / NO_PROFILES_FOUND.

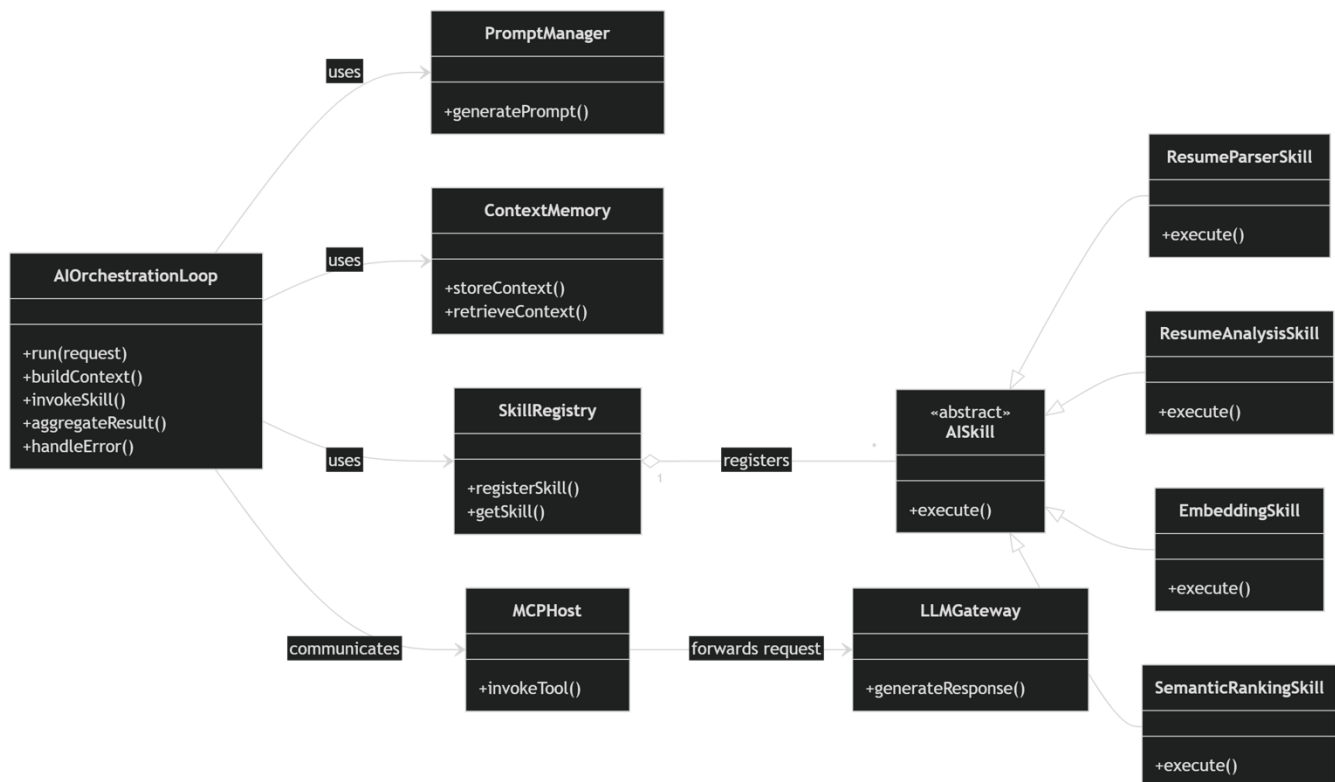
3.2.3 AI Orchestration

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

This class diagram illustrates the collaboration among the AI coordination components. The AIOrchestrationLoop serves as the central coordinator, managing execution context, invoking AI skills through the SkillRegistry, communicating with external AI services via the MCPHost, and aggregating the final response.



3.2.4 Authorization & ABAC Subsystem Class Diagram

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

The following class diagram documents the ABAC (Attribute-Based Access Control) subsystem that provides field-level PII masking. The ABACEngine applies a declarative policy (ABAC_POLICY) to redact sensitive fields based on the requesting user's role.

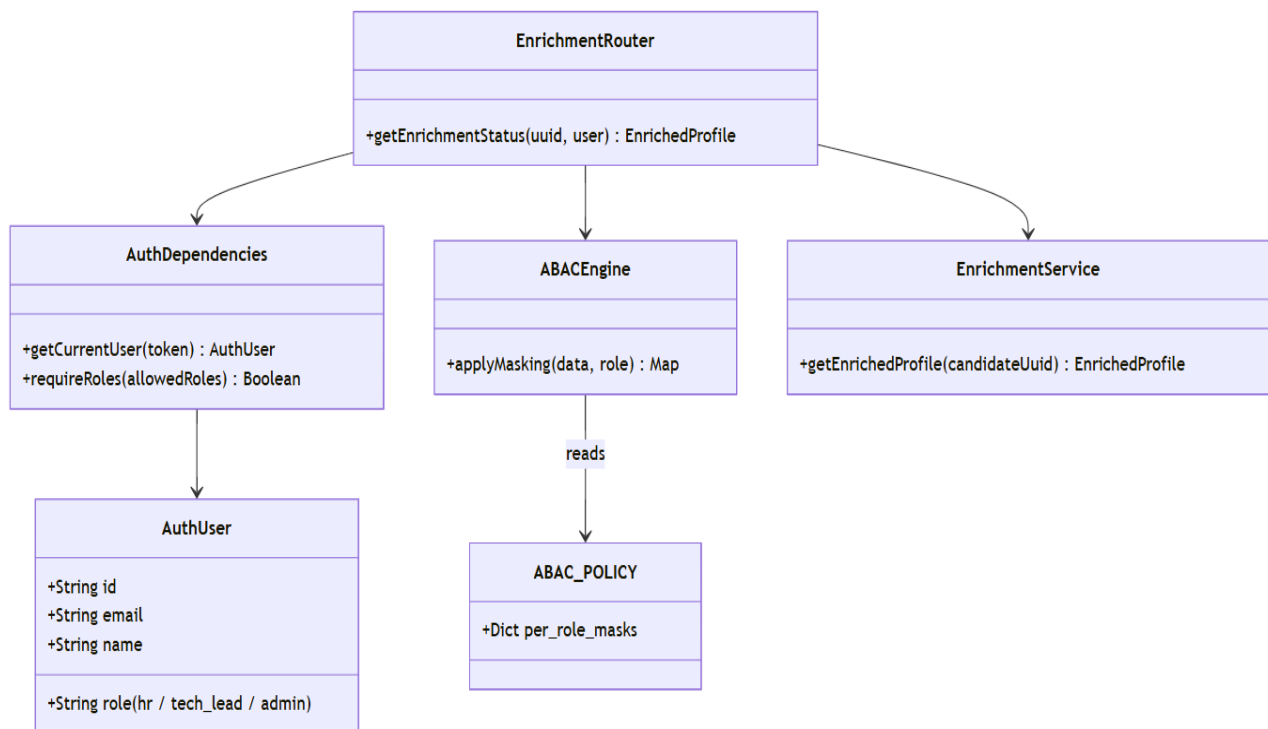
The AuthDependencies middleware resolves the role from the authentication token and passes it through the request chain.

Core classes:

- ABACEngine — Stateless utility that walks the enriched profile data structure and replaces restricted fields with masked values according to ABAC_POLICY.
- AuthDependencies — Provides `getCurrentUser(token)` and `requireRoles(allowedRoles)` dependency functions for FastAPI route guards.
- AuthUser — Value object representing an authenticated user with resolved role (hr, tech_lead, admin).
- ABAC_POLICY — Configuration dict defining which fields are masked per role. For tech_lead: full_name, email, phone, address, salary_expectation.

Integration points:

- EnrichmentRouter — Calls `ABACEngine.applyMasking()` on the enriched profile before returning it to non-privileged roles.
- EnrichmentService — Domain service that produces the enriched profile data consumed by the ABAC engine.



3.2.5 Scheduling Subsystem Class Diagram

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

The following class diagram documents the structural design of the Interview Scheduling Subsystem implemented for the SmartATS Applicant Tracking System. It captures the classes responsible for interviewer management, Google Calendar FreeBusy slot discovery, calendar event creation, and email notification delivery.

Core service and API classes:

- **SchedulingRouter** — Exposes RESTful endpoints for the scheduling workflow: listing available interviewers, querying calendar availability, and confirming interview slots. It delegates all business

logic to the SchedulingService and enforces role-based access control, ensuring only HR and admin users can schedule interviews.

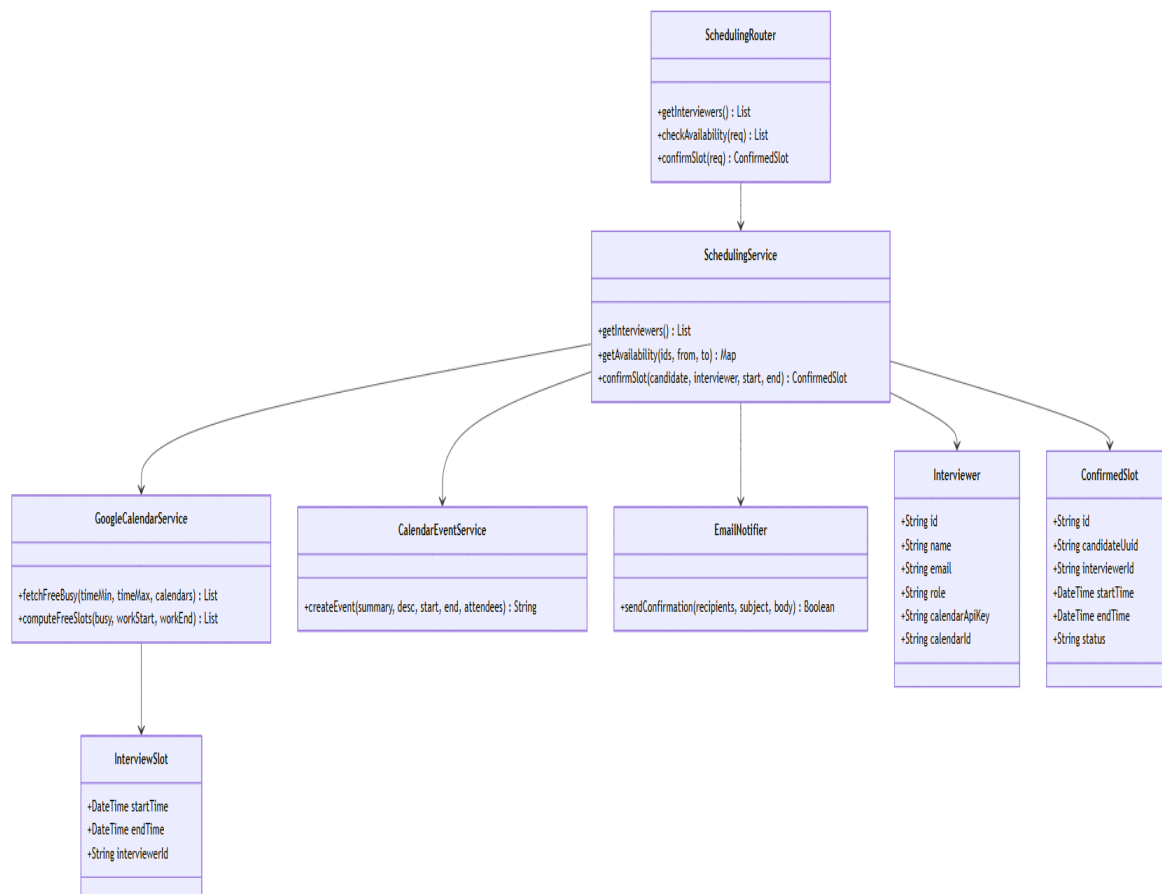
- SchedulingService — Central coordinator for the interview scheduling domain. It manages the interviewer registry, orchestrates FreeBusy queries across multiple calendars, processes slot confirmation requests, and triggers downstream services for calendar event creation and email notification dispatch.
- GoogleCalendarService — Abstraction over the Google Calendar v3 FreeBusy API that retrieves the busy intervals of specified calendars within a given time range. It computes available time slots by intersecting the working hours (07:30–17:00, Monday–Friday) with the returned busy periods, producing a list of free intervals grouped by interviewer.
- CalendarEventService — Handles the creation of calendar events through the Google Calendar events.insert API. Each event includes a summary, description, scheduled time range, attendee list, and reminder configurations. The service constructs the event payload and submits it for creation on the designated calendar.
- EmailNotifier — Manages the dispatch of interview confirmation emails to participants. For each confirmed interview slot, it constructs a plain-text message containing the interview details and sends it via SMTP (smtp.gmail.com:587) with TLS encryption.

Domain / data model classes:

- Interviewer — Models a person who conducts interviews. Each interviewer has a profile (name, role, initials, colour identifier), email

address for notifications, and a Google Calendar configuration (API key and calendar ID) used for availability queries.

- InterviewSlot — Represents a time window during which an interviewer is available. Each slot is produced by the GoogleCalendarService after intersecting busy intervals with the configured working hours.
- ConfirmedSlot — Records an interview that has been successfully scheduled. It captures the candidate reference, interviewer assignment, selected time range, and confirmation status. These records are stored for future reference and auditing.



3.3 Class Specifications

3.3.1 Class Workspace Provider

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Purpose: Central frontend state manager for the resume ingestion workspace, responsible for controlling UI loading workflows, caching local PDF streams, and syncing final AI analytics payloads

Attributes:

Seq	Property	Modifier	Constraint	Description
1	status	private state	Required	Current lifecycle state of the ingestion workspace: IDLE, LOADING, SUCCESS, or ERROR.
2	pdfUrl	private state	Optional	Internal browser Object URL used to display native previews of the uploaded PDF file.
3	analyticData	private state	Optional	Structured candidate profile analytics

				computed and returned by the AI pipeline.
4	errorMessage	private state	Optional	String message storing exception details when an ingestion or socket failure occurs.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	uploadResume(file)	public	Async	Formulates the multipart payload, dispatches the HTTP POST request to the API, and initiates the real-time pipeline.
2	resetWorkspace()	public	Sync	Flushes all structural state parameters and rolls the ingestion interface back to baseline idle mode.
3	_initWebSocketPipeline(job_id)	private	Async Event-driven	Mounts the active WebSocket stream channel to continuously listen for backend transaction pulses.

3.3.2 Class IngestionGateway

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

Purpose: Core backend application orchestrator handling upload streams, magic byte verification, execution job registration, and structural event bridging.

Attributes:

Seq	Property	Modifier	Constraint	Description
1	upload_dir	private	Required	Disk storage path designated for buffering uploaded candidate resume files before extraction.
2	jobs	private	Required	In-memory operational dictionary registry dedicated to indexing and tracking asynchronous ingestion job status states.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	create_job(file, user_id)	public	Async	Commits binary chunks to disk, yields a distinct tracking UUID, establishes user ownership context, and assigns INGESTION_QUEUED status.
2	run_job(job_id, db_session)	public	Async Background	Dispatches execution tasks to the decoupled underlying domain processing layer and fires

				structural milestone signals.
3	<code>_emit_progress(job_id, event_type, data)</code>	private	Async	Updates state tracking vectors and delivers mapped structural frames to the client messaging matrix.

3.3.3 Class Resume Pipeline

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

Purpose: The core functional domain pipeline that sequentializes resume ingestion processing, handling token routing and lifecycle tracking across all specialized AI microservices.

Attributes:

Seq	Property	Modifier	Constraint	Description
1	<code>parser_service</code>	private	Required	Instance of ParserService handling text extraction layers.
2	<code>llm_service</code>	private	Required	Instance of LLMSERVICE translating raw strings into structured formats.
3	<code>embedding_service</code>	private	Required	Instance of EmbeddingService generating numerical vector weights.

Methods:

Seq	Operation	Modifier	Constraint	Description
-----	-----------	----------	------------	-------------

1	process(file_path, user_id, session)	public	Async / Required	Orchestrates the extraction framework step-by-step and persists records into target repository vectors.
2	process_batch(file_paths, session)	public	Async / Optional	Handles multiple pipeline iterations concurrently for batch file arrays.

3.3.4 Class Parser Service

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

- **Purpose:** A stateless utility domain layer in charge of executing content-aware text extraction and sanitation filters over ingested document structures.
- **Attributes:** None (Stateless system module).
- **Methods:**

Seq	Operation	Modifier	Constraint	Description
1	parse(file_path)	public	Sync / Required	Triggers low-level engine primitives to read internal PDF pages and pull layout text strings.
2	cleanup(text)	public	Sync / Required	Applies structural regular expressions to purge broken

				symbols, duplicate line-breaks, and format spacing noise.
--	--	--	--	--

3.3.5 GitHubScraper Class Specification

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127337 - Nguyễn Tiến Cường

Purpose: Encapsulates GitHub API access for repository data, README, and language stats, with caching and fallback handling.

Attributes:

Seq	Property	Modifier	Constraint	Description
1	settings	private	Required	Settings instance holding the GitHub API token.
2	cache_dir	private	Default: <code>“./stored_data”</code>	Directory path for storing cached GitHub data files.
3	cache_ttl_seconds	private	Default: 3600	Time-to-live for cached data, in seconds (1 hour).
4	max_repos	private	Default: 100	Maximum number of repositories fetched per request.
5	timeout_seconds	private	Default: 10.0	HTTP request timeout, in seconds.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	<code>__init__(settings, cache_dir)</code>	public	Required	Initialises the scraper; creates the cache directory if missing.
2	<code>fetch_profile(github_username)</code>	public	Async, Required	Cache-first fetch of the GitHub profile; falls back to a live call on miss/expiry.

3	<code>_fetch_from_cache(username)</code>	private	Async, Optional	Loads a cached profile from JSON; returns None if missing/corrupted/expired.
4	<code>_fetch_from_api(username)</code>	private	Async, Required	Performs the live GitHub API call with the auth token when available; handles rate limits.
5	<code>_parse_repos(repos_data)</code>	private	Sync, Required	Maps the raw API response into GitHubRepo objects (name, language, size).
6	<code>_calculate_top_languages(repos)</code>	private	Sync, Required	Computes the top 3 languages by code size as percentages.
7	<code>_fetch_readme_content(username, repos)</code>	private	Async, Optional	Fetches README.md from up to 5 recent repos and concatenates it.
8	<code>_save_to_cache(username, profile)</code>	private	Sync, Optional	Persists the fetched profile to the local cache with a TTL timestamp.
9	<code>_build_headers()</code>	private	Sync, Required	Builds HTTP headers, adding a Bearer token when configured.
10	<code>invalidate_cache(username)</code>	public	Sync, Optional	Deletes cached data for a username to force a fresh fetch.

Exception handling:

- **GitHubAPIException** – non-2xx response or rate limit exceeded.
- **CacheException** – cache file read/write fails or is corrupted.
- **NetworkException** – HTTP client network errors (timeouts, connection failures).

Design considerations:

- **Cache-first strategy** – checks the local cache before calling the API.
- **Async/await** – all I/O is asynchronous to avoid blocking the event loop.
- **Rate-limit awareness** – respects GitHub's 5000/hr (authenticated) and 60/hr (unauthenticated) limits.
- **Graceful degradation** – returns partial data when part of the fetch fails, e.g. README fetch fails but repository data still succeeds.
- **Thread safety** – cache file operations use file locking under concurrent access.

3.3.6 LinkedIn Scraper Service Class

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

Purpose: Acquires a candidate's LinkedIn profile via using the Apify API Store , with a Playwright browser-automation fallback that converts the profile page to Markdown.

Inheritance: None (standalone utility class).

Attributes:

Seq	Property	Modifier	Constraint	Description
1	settings	private	Required	Settings instance holding apify_api_token and linkedin_cookie.
2	playwright	private	Required	Active Playwright instance used to drive the headless browser.
3	browser	private	Required	Browser instance (Chromium) launched for scraping sessions.
4	context	private	Required	Directory used to cache LinkedIn JSON payloads per username.
5	cache_dir	private	Default: “./stored_data”	Directory used to cache scraped/actor LinkedIn JSON payloads per username.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	__init__(settings)	public	Required	Stores config; browser/context are created lazily via _init_browser().
2	_init_browser()	private	Async, Required	Launches Playwright/Chromium and creates an authenticated

				context.
3	fetch_profile(linkedin_url, candidate_uuid, settings)	public	Async, Required	Normalises the URL, checks cache, then calls Apify actor GOvL4O4RwFqsdIqXF; maps the response into a LinkedInProfile.
4	scrape_to_markdown(linkedin_url)	public	Async, Optional	Fallback: navigates via Playwright and returns the page as Markdown, or None on failure.
5	_save_to_cache(username, profile)	private	Sync, Optional	Persists the LinkedInProfile as JSON to avoid repeated Apify calls.
6	close()	public	Async, Required	Closes the BrowserContext, Browser, and Playwright instance.

Exception handling:

- **ApifyActorException** – actor run fails, returns empty data, or the token is missing.
- **PlaywrightNavigationException** – fallback browser can't load/parse the profile page.
- **CacheException** – local JSON cache can't be read or written.

Design considerations:

- **Cache-first strategy** – checks the local JSON cache before calling Apify.
- **Proxy support** – Apify's managed proxy infrastructure reduces LinkedIn IP blocking.
- **Graceful, deterministic fallback** – on Apify failure, fetch_linkedin_profile returns a structured fallback LinkedInProfile instead of failing the pipeline.
- **URL normalisation** – LinkedIn URLs are standardised before use as a cache key.
- **Resource cleanup** – close() must always run to avoid leaking browser processes.

3.3.7 Supporting Domain Model Classes

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

The table below lists the remaining classes that complete the Data Crawling Subsystem.

Class	Description
EnrichmentService	Central orchestrator: fetches GitHub/LinkedIn data, generates analytics, applies the idempotent merge, and updates enrichment status.
GitHubProfile / GitHubRepo	GitHub profile aggregate (public_repos_count, top_languages, readme_content, repos) and per-repository metadata (name, language, size).
LinkedInProfile	LinkedIn profile aggregate holding experiences, educations, certifications, full_name, headline, profile_url and avatar_url.
LinkedInExperience LinkedInEducation LinkedInCertification	Individual work, education, and certification records scraped from LinkedIn.
MockAnalytics TechnicalSkillMatrix	Derived analytics: match_confidence_score, score_increase, semantic_tags, and the pre-/post-enrichment skill matrix.
EnrichedProfile	Aggregate root combining GitHubProfile, LinkedInProfile and MockAnalytics for one candidate.
CandidateEnrichment EnrichmentStatus	Tracks the lifecycle of an enrichment job (QUEUED → IN_PROGRESS → ENRICHED / ENRICHMENT_FAILED / NO_PROFILES_FOUND).
CandidateRepository	In-memory persistence gateway for candidate records and raw GitHub/LinkedIn JSON payloads.
LinkedInScraperService	Playwright-driven browser automation used as a fallback scraper, converting a profile page to Markdown.

3.3.8 AI Orchestration Class

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Purpose: Coordinates the execution of AI workflows by managing context, selecting AI skills, invoking LLM services, and aggregating results into a unified response.

Inheritance: None

Attributes

Name	Type	Description
skillRegistry	SkillRegistry	Stores all available AI skills
promptManager	PromptManager	Builds prompts for LLM
contextMemory	ContextMemory	Maintains execution context
mcpHost	MCPHost	Connects to external AI tools
llmGateway	LLMGateway	Sends requests to LLM

Methods

Method	Description
run(request)	Executes the orchestration workflow
buildContext()	Builds contextual information for AI
invokeSkill()	Invokes a registered AI skill
aggregateResult()	Merges outputs from different skills
handleError()	Handles execution exceptions

3.3.9 Scheduling Service Class

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

Purpose: Central coordinator for the interview scheduling workflow. Manages the interviewer registry, orchestrates FreeBusy queries via GoogleCalendarService, and coordinates event creation and email dispatch.

Inheritance: None (standalone service class).

Attributes

Seq	Property	Modifier	Constraint	Description
1	_interviewers	private	Required	In-memory list of available interviewers loaded from

				interviewers.json.
2	_confirmed_slots	private	Required	In-memory dictionary of confirmed interview slots, backed by confirmed_slots.json.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	getInterviewers()	public	Sync	Returns the list of available interviewers with their connection status.
2	getAvailability(interviewerIds, dateFrom, dateTo)	public	Async	Iterates over each interviewer and returns aggregated free slots within working hours.
3	confirmSlot(candidateUuid, interviewerId, startTime, endTime)	public	Async	Creates calendar event, sends email, and persists the confirmed slot.

3.3.10 Google Calendar Service Class

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127382 - Trần Nhật Hoàng

Purpose: Abstraction layer over the Google Calendar v3 FreeBusy API. Retrieves busy intervals for specified calendars and computes free slots within working hours.

Inheritance: None (standalone infrastructure service).

Attributes:

Seq	Property	Modifier	Constraint	Description
1	WORK_START	constant	07:30	Start of working hours for scheduling.
2	WORK_END	constant	17:00	End of working hours for scheduling.
3	WORKING_DAYS	constant	Monday–Friday	Working days for interview scheduling.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	fetchFreeBusy(timeMin, timeMax, calendars)	public	Async	Sends request to Google Calendar FreeBusy API, returns busy intervals for each calendar.
2	computeFreeSlots(busyIntervals)	public	Sync	Intersects busy periods with working hours to produce free time windows.

3.3.11 Class CalendarEventService

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Purpose: Handles creation of interview calendar events via the Google Calendar events.insert API. Builds event payload with summary, description, time range, and attendee list.

Inheritance: None (standalone infrastructure service).

Methods:

Seq	Operation	Modifier	Constraint	Description
1	createEvent(summary, description, startTime, endTime, attendees)	public	Async	Submits event to Google Calendar and returns the created event ID.

3.3.12 Class EmailNotifier

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

Purpose: Manages dispatch of interview confirmation emails to all participants via SMTP (smtp.gmail.com:587). Reads SMTP credentials from environment variables.

Inheritance: None (standalone notification service).

Methods:

Seq	Operation	Modifier	Constraint	Description
1	sendConfirmation(recipients, subject, body)	public	Async	Builds formatted email and sends to each recipient through the configured SMTP relay.

3.3.13 Class ABACEngine

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127382 - Trần Nhật Hoàng

Purpose: Provides field-level PII masking using Attribute-Based Access Control. Walks the enriched profile data structure and replaces restricted fields with masked values based on the user role.

Inheritance: None (stateless utility class).

Attributes:

Seq	Property	Modifier	Constraint	Description
1	ABAC_POLICY	static final	Required	Configuration defining which fields are masked per role. tech_lead redacts full_name, email, phone, address, salary_expectation.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	applyMasking(data, role)	public	Sync	Recursively examines each field; masks restricted fields, preserves others; returns the masked data object.

3.3.14 Class AuthDependencies

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Purpose: FastAPI dependency provider for authentication and authorization. Resolves current user from token and guards endpoints based on allowed roles.

Inheritance: None (collection of dependency functions).

Methods:

Seq	Operation	Modifier	Constraint	Description
1	getCurrentUser(token)	public	Sync	Validates token and returns AuthUser with resolved role (hr, tech_lead, admin).
2	requireRoles(allowedRoles)	public	Sync	Guard that verifies current user role is in allowed list; raises error if unauthorized.

4 Data Design

4.1 Data Diagram

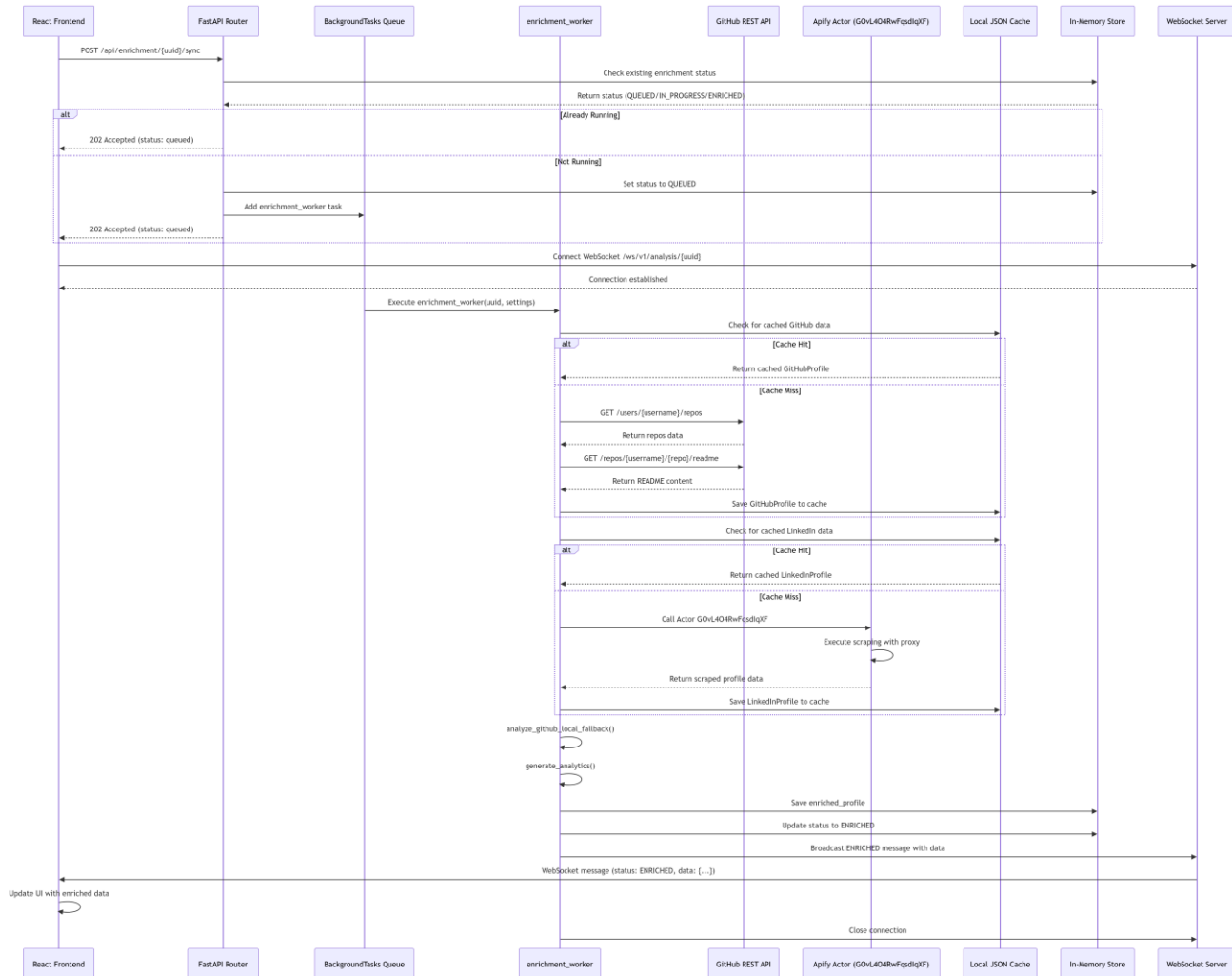
4.1.1 *Asynchronous Crawling Architecture – Enrichment Data Flow*

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường

The Profile Enrichment Module combines FastAPI BackgroundTasks with a WebSocket channel, keeping API responses non-blocking while still giving recruiters live progress feedback.



Link picture:

https://drive.google.com/file/d/1B7mpe0MZi8azuxWbfz4Z1cGSXTMJGQVW/view?usp=drive_link

Enrichment sequence (client → API → background worker → external sources → WebSocket):

- 1. The React frontend sends POST /api/enrichment/{uuid}/sync; the API checks the existing enrichment status in the in-memory store.
- 2. If already QUEUED/IN_PROGRESS, the API returns 202 immediately; otherwise it sets status QUEUED, schedules enrichment_worker, and returns 202.
- 3. The frontend opens a WebSocket connection to /ws/v1/analysis/{uuid} to receive the final result.
- 4. The worker checks the GitHub cache; on a miss it calls the GitHub REST API and writes the result back to cache.
- 5. Same cache-first pattern for LinkedIn, via the Apify actor (GOvL4O4RwFqsdIqXF) through a proxy.

- 6. It runs the local GitHub fallback analysis and `generate_analytics()`, merges the result via `idempotent_merge()`, and saves it with status ENRICHED.
- 7. The worker broadcasts ENRICHED (or ENRICHMENT_FAILED) to all subscribed WebSocket clients, and the frontend updates accordingly.

Key backend components:

- **FastAPI BackgroundTasks queue** – `sync_candidate_profile()` deduplicates in-flight jobs before queuing `enrichment_worker`, and always returns HTTP 202 immediately.
- **`enrichment_worker()`** – orchestrates cache-first reads, live-fetch fallback, analytics generation, idempotent merge, and WebSocket notification; failures are reported as ENRICHMENT_FAILED.
- **Apify actor integration (`fetch_linkedin_profile`)** – cache-first, then calls Apify actor `GOvL4O4RwFqsdIqXF` with proxy support; on failure returns a deterministic fallback profile.
- **WebSocket endpoint (`websocket_endpoint`)** – returns the cached result immediately if already finished, otherwise registers the socket for a later broadcast and cleans up on disconnect.

Performance optimisations:

- Cache-first strategy reduces external API calls.
- GitHub and LinkedIn fetches can be parallelised (currently sequential).
- HTTPX AsyncClient connection pooling is reused across GitHub calls.
- Apify's proxy infrastructure rotates IPs to reduce scraping blocks.
- The WebSocket only pushes the final result, not intermediate progress.

Error handling strategy:

Error Type	Detection	Recovery Mechanism	User Impact
GitHub API rate limit	HTTP 403 response	Use cached data or return a partial profile	Delayed enrichment, partial data
Apify actor failure	Empty dataset or exception	Return a structured fallback mock profile	Full profile with mock LinkedIn data
Network timeout	HTTPX timeout exception	Retry with exponential backoff	Slight delay, eventual success
Cache corruption	JSON parse exception	Delete the cache file and refetch	One-time delay, transparent to the user
WebSocket	onclose event	Client-side reconnection with HTTP status polling	Brief interruption,

disconnect

fallback

automatic recovery

4.1.2 Real-time Document Ingestion and Verification Data Flow

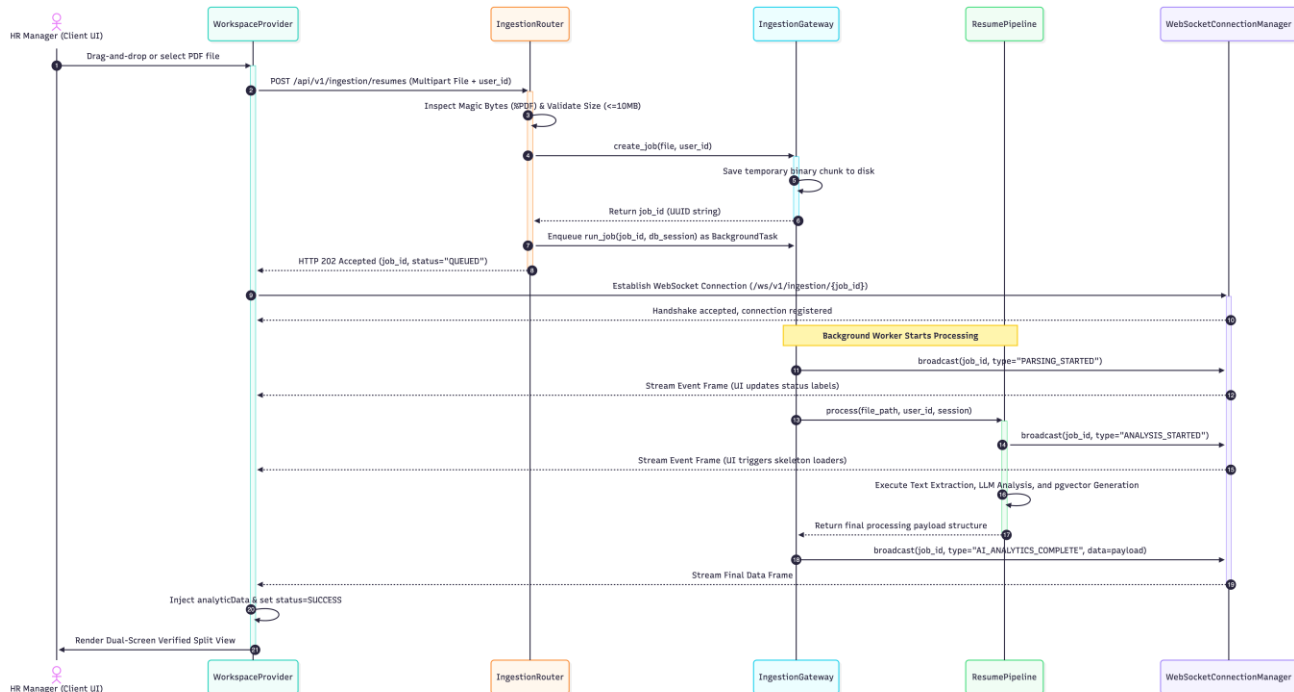
Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

The Ingestion Module establishes a non-blocking asynchronous data flow utilizing FastAPI Background Tasks coordinated with an active time-series WebSocket channel. This deployment ensures immediate transaction responses while piping real-time telemetry updates to the recruiter's verification workspace.

Ingestion Subsystem Sequence Diagram



- **Asynchronous Offloading:** The IngestionRouter delegates the resource-intensive AI workflow to the backend worker as a background task immediately after verifying basic file criteria, shifting the operational response to a minimal latency environment.
- **Granular Telemetry Streaming:** Instead of relying on rigid polling infrastructure, the shared WebSocketConnectionManager broadcasts distinct lifecycle signals (PARSING_STARTED,

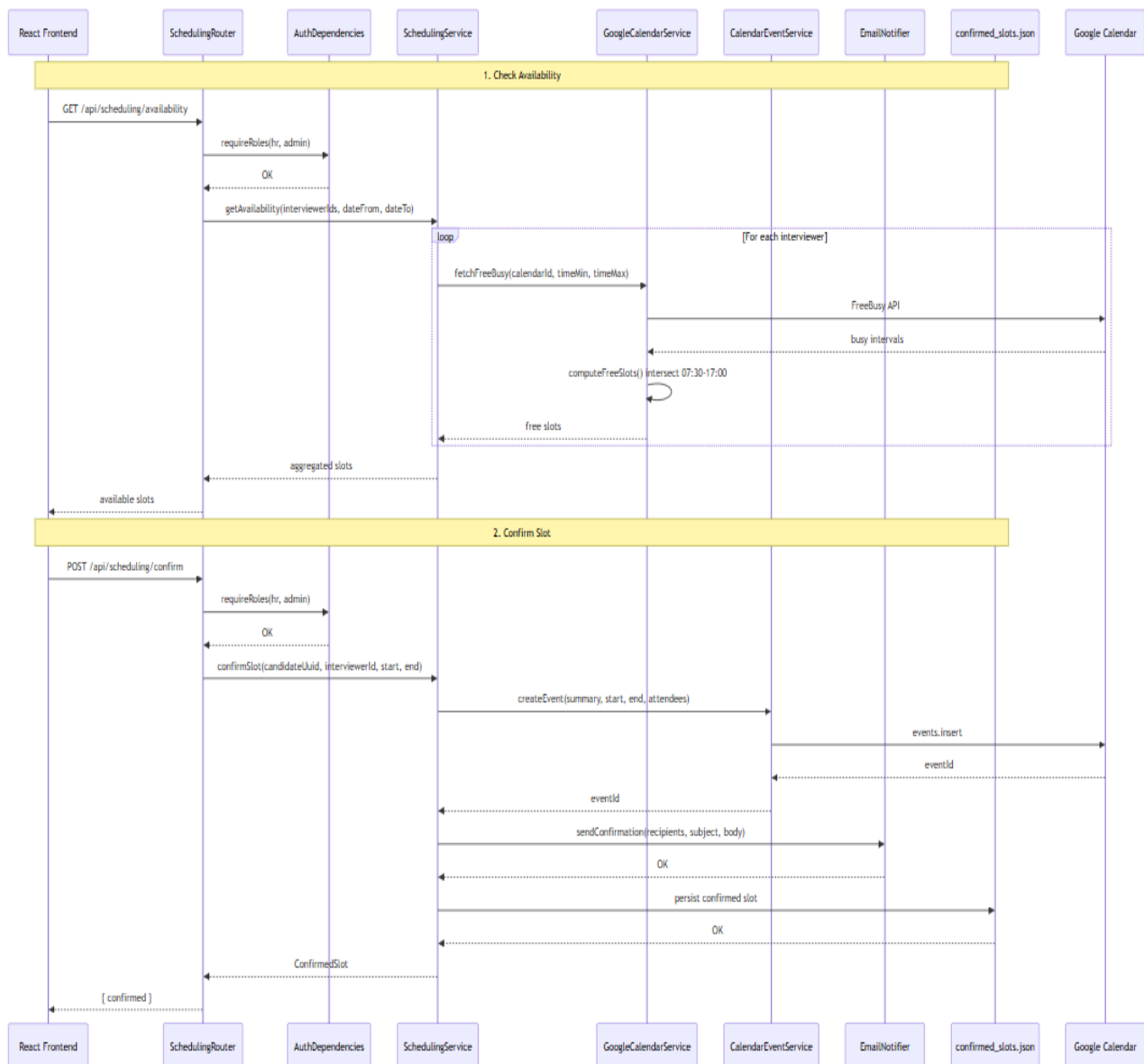
ANALYSIS_STARTED), allowing the client viewport to dynamically toggle visual skeleton layers based on background milestone execution.

4.1.3 Interview Scheduling Data Flow

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng



The Interview Scheduling flow follows a linear sequence:

- 1. The HR user selects interviewers and requests availability via the frontend scheduling panel.
- 2. The SchedulingService iterates over selected interviewers and calls `GoogleCalendarService.fetchFreeBusy()` for each.
- 3. `GoogleCalendarService` queries the Google Calendar FreeBusy API with the interviewer's calendar ID and API key.
- 4. The busy intervals are returned to `SchedulingService`, which calls `computeFreeSlots()` to produce available time windows within working hours (07:30-17:00, Mon-Fri).
- 5. Aggregated availability is returned to the frontend and displayed in the centre panel.
- 6. The HR user selects a slot and confirms the interview.
- 7. `SchedulingService` calls `CalendarEventService.createEvent()` to create the calendar event.
- 8. `SchedulingService` calls `EmailNotifier.sendConfirmation()` to send email notifications to all participants.
- 9. The confirmed slot is persisted to `confirmed_slots.json`.

Authentication and authorisation:

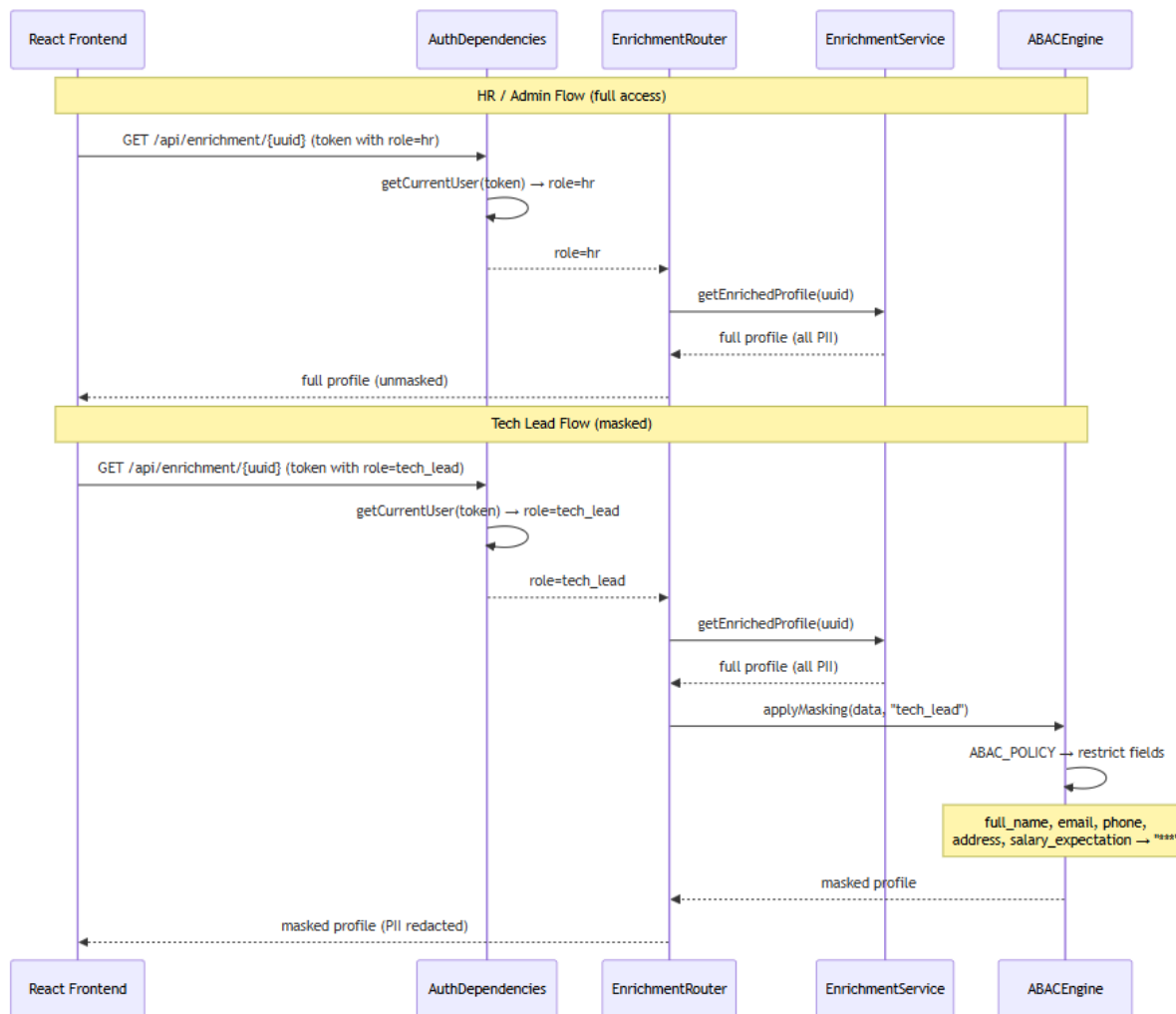
Every scheduling endpoint is guarded by `AuthDependencies.requireRoles(["hr", "admin"])`. Non-privileged roles (tech_lead) receive a 403 Forbidden error.

4.1.4 ABAC Masking Data Flow

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn



The ABAC masking data flow describes how enriched candidate profile data passes through the ABAC engine before reaching the frontend. The sequence begins when a user requests an enriched profile via the `GET /api/enrichment/{uuid}` endpoint. The AuthDependencies middleware resolves the user role from the authentication token. The EnrichmentRouter then retrieves the full enriched profile from EnrichmentService. If the resolved role is `tech_lead`, the `ABACEngine.applyMasking()` method is invoked to redact PII fields before the response is returned. If the role is `hr` or `admin`, the full unmasked profile is returned directly.

The masking flow follows these steps:

1. Frontend sends GET /api/enrichment/{uuid} with a bearer token containing the user role.
2. AuthDependencies.getCurrentUser() validates the token and resolves the role (hr, tech_lead, or admin).
3. EnrichmentService retrieves the full enriched profile (including all PII fields) from the in-memory store or JSON cache.
4. EnrichmentRouter checks the resolved role: if tech_lead, it calls ABACEngine.applyMasking(data, "tech_lead") before returning the response.
5. ABACEngine walks the enriched profile data structure and replaces restricted fields (full_name, email, phone, address, salary_expectation) with masked values.
6. The masked profile is returned to the frontend, where PII fields display "***" placeholder text.
7. For hr or admin roles, the full unmasked profile is returned with no transformations applied.

ABAC policy configuration:

- ABAC_POLICY is a static dictionary mapping role names to lists of restricted field paths. Currently only tech_lead has restrictions; hr and admin have empty restriction lists.
- Field paths are dot-notation strings (e.g., "personal_info.full_name") that the ABACEngine recursively resolves against the profile JSON structure.

4.2 Data Specification

4.2.1. *Idempotent Merge Strategy for Enriched Candidate Profiles*

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

Repeated enrichment runs for the same candidate must converge on a consistent state without duplicating or losing data. The `idempotent_merge(existing, new_data)` function guarantees this by combining profile fields using a deterministic strategy tailored to each attribute type before persisting the result via `SupabaseRepository`.

Merge rules by field:

Field	Merge Strategy	Rationale
github	Prefer new_data, fallback to existing	GitHub repository data is time-sensitive; the latest API response reflects current code activity.
linkedin.full_name / headline / profile_url / avatar_url	Prefer new_data, fallback to existing	Profile metadata evolves as candidates update their professional profiles over time.
linkedin.experiences	De-duplicate by company, keep new_data's entry	Company name serves as a unique key (<code>{**existing_exp, **new_exp}</code>); newer entries overwrite older entries for the same employer.
linkedin.educations / certifications	Prefer new_data, fallback to existing	Educational background is generally stable, but new data fetches may append newly earned qualifications.
analytics.semantic_tags	Union of existing and new tags, capped at 8	Tags accumulate across runs so no previously extracted technical skill tag is lost.
analytics.technical_skill_matrix (pre/post)	Maximum of existing and new, per skill	Both baseline and enriched radar scores retain the highest competency evidence observed across runs.
analytics.match_confidence_score	Maximum of existing and new_data	Preserves the highest confidence score (<code>max()</code>) derived from cross-channel verification.
analytics.score_increase	Maximum of existing and new	Reflects the maximum positive delta achieved through multi-channel data enrichment.

github_username / linkedin_url	Prefer new_data, fallback to existing	Accounts for profile link corrections or updates made between sync requests.
full_name	new_data.full_name → new_data.linkedin.full_name → existing.full_name	Cascading fallback keeps a name available whenever any source has one.

1. Idempotence Guarantees

Safe Multiple Invocations: Executing enrichment repeatedly for the same candidate_uuid converges on a consistent state without duplicating rows/data via UPSERT ON CONFLICT (candidate_uuid).

Commutative Merge: Merging order does not affect output (unions tags, selects maximums for scores/radar matrices, and prioritizes new_data for metadata).

Associative Operations: Sequential updates compose cleanly, ensuring (A" merge " B)" merge " C=A" merge " (B" merge " C) without state corruption.

Idempotent Identity: Merging a profile with itself (X" merge " X=X) produces zero side effects.

2. Persistence Strategy (Supabase PostgreSQL Implementation)

Candidate enrichment data is persisted across three core PostgreSQL tables:

github_profiles: Stores GitHub repositories, languages, and README metadata.

linkedin_profiles: Stores work experience, education, and credentials.

candidate_enrichments: Stores merged enriched_profile JSONB documents and status flags (QUEUED, IN_PROGRESS, ENRICHED, ENRICHMENT_FAILED, NO_PROFILES_FOUND).

Example Supabase PostgreSQL Schema (candidate_enrichments Table)

```
CREATE TABLE candidate_enrichments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  candidate_uuid VARCHAR(255) UNIQUE NOT NULL,
  enrichment_status VARCHAR(50) NOT NULL
  CHECK (enrichment_status IN ('QUEUED', 'IN_PROGRESS', 'ENRICHED', 'ENRICHMENT_FAILED',
'NO_PROFILES_FOUND')),
  enriched_profile JSONB,
  github_username VARCHAR(255),
  linkedin_url TEXT
  full_name VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- GIN index for high-performance JSONB document querying
CREATE INDEX idx_candidate_enrichments_profile_gin
ON candidate_enrichments USING GIN(enriched_profile);
```

3. Performance & Production Trade-offs

- **Performance Optimizations:**

- **GIN Indexing:** Enables sub-millisecond JSONB attribute lookups and tag filtering.
- **In-Memory Caching & Connection Pooling:** Fast WebSocket broadcasting before DB writes and minimized query latency.

- **Limitations & Future Enhancements:**

- *Current Limit:* Sequential API scraping and un-persisted BackgroundTasks queue.
- *Production Fix:* Migrate to Celery/Redis for persistent retries and distributed caching.

4.2.2. Scheduling Data Specification

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

The Scheduling Module uses JSON files for persistent storage of interviewer profiles and confirmed interview slots. The data structures are defined as follows.

1. `interviewers.json` — Seed data loaded on first access:

- `id`: string — Unique identifier (e.g., "interviewer-sarah-chen")
- `name`: string — Display name
- `email`: string — Email address (may be empty if not configured)
- `role`: string — Job title (e.g., "Staff Engineer", "Tech Lead")
- `initials`: string — Avatar initials
- `color`: string — Hex colour code for UI identification
- `cal_connected`: bool — Whether calendar API key is provided
- `calendar_api_key`: string — Google Calendar API key
- `calendar_id`: string — Calendar identifier (default: "primary")

2. `confirmed_slots.json` — Persisted after each confirmation:

- `id`: string — UUID generated on confirmation
- `candidate_uuid`: string — Reference to the candidate
- `interviewer_id`: string — Reference to the interviewer
- `start_time`: string — ISO 8601 datetime
- `end_time`: string — ISO 8601 datetime
- `status`: string — "confirmed" (future: "cancelled", "rescheduled")
- `created_at`: string — ISO 8601 timestamp of creation

Persistence strategy — current implementation:

Interviewer profiles are seeded from a JSON file on first access and kept in memory for fast lookups.

Confirmed interview records are stored as a JSON object keyed by record ID, loaded at startup and persisted on each confirmation.

All scheduling data is kept in memory for fast access, with file-based persistence ensuring durability across restarts.

Known limitations and recommended enhancements for production:

Current limitations — File-based persistence lacks transactional guarantees; no support for rescheduling or cancellation workflows in the current version.

Recommended enhancements — Migrate to PostgreSQL for confirmed slots with proper transactional support; add Redis caching for availability queries; implement reschedule and cancel endpoints.

4.2.3. *ABAC Data Specification*

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

The ABAC subsystem uses a configuration-based policy definition and a simple user data model for role resolution. The data structures are defined as follows.

1. ABAC_POLICY — Static configuration defining field-level masking rules per role:

role: string — Role identifier (hr, tech_lead, admin).

restricted_fields: list[string] — Dot-notation field paths to mask. For tech_lead: [“full_name”, “email”, “phone”, “address”, “salary_expectation”].

Policy is stored as a Python dictionary constant (ABAC_POLICY) in abac.py and loaded at runtime.

2. AuthUser — Value object representing an authenticated user:

id: string — User identifier.

email: string — User email address.

name: string — Display name.

role: string — Resolved role (hr, tech_lead, admin).

3. Masked field format:

When a field is restricted, its value is replaced with the string “***” (three asterisks) in the API response JSON.

Non-restricted fields are returned unchanged. The structure of the JSON object is preserved (fields are not removed, only their values are replaced).

Persistence strategy:

ABAC_POLICY is defined as a static configuration in the source code (modules/shared/infrastructure/abac.py) and does not require database persistence.

AuthUser objects are constructed per-request from the authentication token and are not persisted.

Role information is encoded in the dummy token format: dummy-access-token::{role}.

5 User Interface and User Experience Design

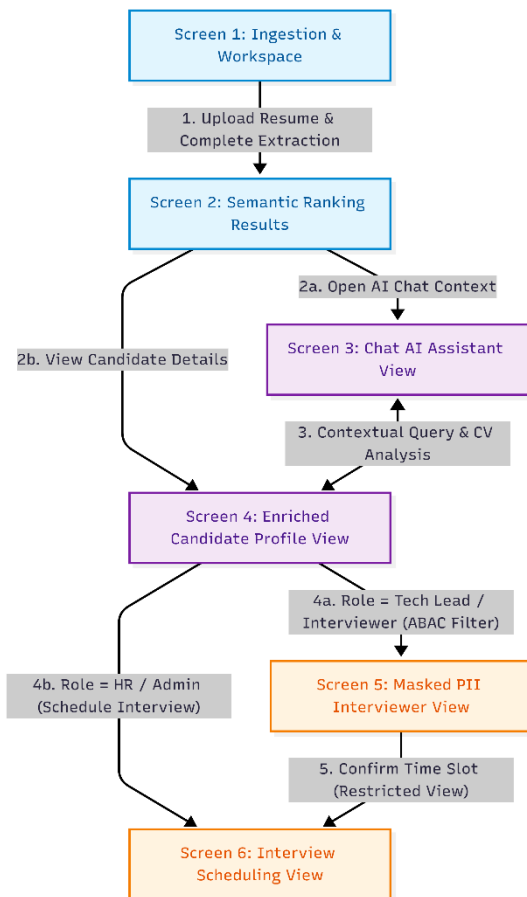
5.1. Screen Diagram

5.1.1. Global Screen Navigation Matrix

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường



Seq	Screen Name	Description & Primary Actions
1	Ingestion & Workspace View	Dual-panel workspace enabling drag-and-drop resume uploading, real-time extraction telemetry streaming, and side-by-side verification.

Seq	Screen Name	Description & Primary Actions
2	Semantic Ranking Results	Ranked candidate list matched against job descriptions via vector similarity with dynamic skill threshold filtering.
3	Chat AI Assistant View	Interactive LLM conversational workspace contextually bound to candidate profiles for natural language querying.
4	Enriched Candidate Profile View	Comprehensive profile view aggregating external GitHub repositories, LinkedIn experience, and radar charts.
5	Masked PII Interviewer View	ABAC security-enforced view dynamically redacting sensitive candidate personal data for technical interviewers.
6	Interview Scheduling View	Three-panel calendar interface integrating Google Calendar API for FreeBusy lookup and automated interview booking.

5.1.2. AI Orchestration

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn



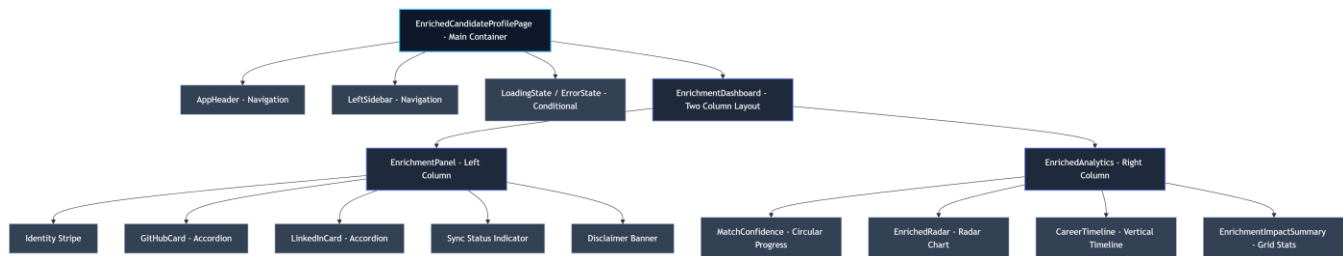
Seq	Screen	Description
1	Search Screen	Enter job description or search criteria.
2	Semantic Ranking Results	Display ranked candidate resumes based on semantic similarity.
3	Chat AI Assistant View	Interactive AI assistant for resume analysis and candidate comparison.

5.1.3. Enriched Candidate Profile View: Component Hierarchy

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng



Link Picture:

https://drive.google.com/file/d/1PFmqQLEXu3tZ4GgfNBOlwmgcsUN_kRu3/view?usp=sharing

Seq	Screen	Description
1	EnrichedCandidateProfilePage	Top-level container page managing candidate state, WebSocket streaming, and main layout structure.
2	EnrichmentPanel (Left Column)	Left column container consolidating external platform integrations (GitHubCard repo stats & LinkedInCard work history).
3	EnrichedAnalytics (Right Column)	Right column container displaying AI match metrics, circular MatchConfidence score, and EnrichedRadar skill chart.
4	CareerTimeline	Vertical chronological timeline visualizing verified candidate work experience and academic milestones.

5.2. Screen Specifications

5.2.1. Screen 1 — Ingestion & Workspace View

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

1. Context & State Management

State Variable	Data Type	Initial Value	Description
status	String	"IDLE"	Workspace lifecycle state: "IDLE", "LOADING", "SUCCESS", or "ERROR".
pdfUrl	String null	null	Local browser Object URL (blob:...) for native PDF preview rendering.
analyticData	Object null	null	Structured candidate profile JSON object received upon extraction completion.
errorMessage	String null	null	Captured exception string rendered when network or extraction fails.
jobId	String null	null	Active tracking UUID issued by backend gateway for WebSocket subscription.
wsRef	Ref<WebSocket>	null	Mutable React ref holding the active WebSocket connection instance.

2. Target UI Event Handling Matrix

Target UI Event	Triggered Handler	Functional Logic & Processing Details
-----------------	-------------------	---------------------------------------

Target UI Event	Triggered Handler	Functional Logic & Processing Details
File Drag & Drop	uploadResume(file)	1. Shift UI state to status = LOADING and clear prior error messages. 2. Generate local object URL via URL.createObjectURL(file) for PDF toolbar previewing. 3. Formulate FormData payload and dispatch HTTP POST to /api/v1/ingestion/resumes.
Upload Accepted	onUploadSuccess(job_id)	1. Extract transaction job_id from the HTTP 202 Accepted response packet. 2. Execute _initWebSocketPipeline(job_id) to initiate WebSocket handshake.
Telemetry Dispatched	ws.onmessage(event)	1. Parse inbound JSON frame and check structural type indicator. 2. If type is PARSING_STARTED or ANALYSIS_STARTED, update step labels dynamically on screen.
Extraction Completed	ws.onmessage(type=AI_ANALYTICS_COMPLETED)	1. Extract structured candidate analysis payload from event message. 2. Inject payload directly into local analyticData state. 3. Set status = SUCCESS to remove skeleton loaders and show verified split-view. 4. Cleanly close the active WebSocket connection.
Pipeline Failure	ws.onerror() / type=INGESTION_FAILED	1. Capture error payload and write to errorMessage register. 2. Set status = ERROR to render fallback wizard for

Target UI Event	Triggered Handler	Functional Logic & Processing Details
		manual correction.

5.2.2. Screen 2 - Specification and Event Handling

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Screen Context: Loading / Ingestion Progress Screen

Target UI Event	Triggered Handler	Functional Logic & Processing Details
Ingestion Pipeline Triggered	uploadResume(file)	1. Shift interface layer state flag to status = LOADING and clear historical exception fields. 2. Build a local virtual link identifier via URL.createObjectURL(file) and feed it directly to the PdfToolbar rendering instance. 3. Wrap the file payload inside a standard FormData stream object to initiate API submission.
Upload Endpoint Accepts Request	onUploadSuccess(job_id)	1. Parse the inbound validation packet response to extract the distinct transaction job_id issued by the gateway. 2. Call the background context method _initWebSocketPipeline(job_id) to subscribe to the state routing port.
Telemetry Pulse Dispatched by	ws.onmessage(event)	1. De-serialize the incoming JSON transaction text frame and validate the structural type indicator field.

WS		2. If the target indicator reads PARSING_STARTED or ANALYSIS_STARTED, adjust step sub-labels dynamically on the UI to present accurate progress tracking to the HR Manager.
AI Processing Complete Frame Received	ws.onmessage(event) with type = AI_ANALYTICS_COMPLETE	1. Extract the dense structured candidate analysis profiles and criteria blocks embedded inside the incoming data field. 2. Map and inject the data block directly into the local frontend analyticData state dictionary. 3. Advance the master application layout state to status = SUCCESS, lifting the loading skeletons to expose data visualization assets. 4. Cleanly terminate the WebSocket handle connection instance to optimize resource channels.
Pipeline Failure / Channel Severed	ws.onerror() or type = INGESTION_FAILED	1. Catch the operational exception data and compile it inside the global errorMessage register. 2. Adjust workspace properties to status = ERROR, prompting the interface layer to render the FallbackDataWizard layout context for manual structural entry overrides.

5.2.3. Screen 3 - Semantic Ranking Results

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127337 - Nguyễn Tiến Cường

Target UI Event	Triggered Handler	Functional Logic & Processing Details
Click Search	searchCandidates()	Generate embedding for job description and perform semantic search.

Select Candidate	viewCandidate()	Display detailed resume information.
Click Chat AI	openAssistant()	Open AI Assistant with selected candidate context.
Apply Filter	filterResults()	Filter ranked candidates by experience, skills, or score.
Sort Results	sortResults()	Sort candidates by similarity score or other criteria.

- **Components**

Component	Description
Search Bar	Input job description
Ranking Table	Ranked candidates
Similarity Score	Matching score
Candidate Summary	AI-generated summary
Filter Panel	Skills / Experience
Chat AI Button	Open AI Assistant

5.2.4. Screen 4 - Chat AI Assistant View

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

Target UI Event	Triggered Handler	Functional Logic & Processing Details
Send Message	sendPrompt()	Send recruiter query to AI orchestration loop.
Select Suggested Question	selectSuggestion()	Populate predefined prompt.
Change Candidate	loadCandidateContext()	Update AI context with selected resume.
Copy Response	copyResponse()	Copy AI-generated response.
Return	backToRanking()	Navigate back to Semantic Ranking Results.

- **Component**

Component	Description
Chat Window	Display conversation
Message Input	Enter prompt
Suggested Questions	Common recruiter questions

Candidate Context Panel	Selected candidate information
AI Response Panel	Generated response

5.2.5. Screen 5 - Enriched Candidate Profile View State management:

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

Variable	Type	Initial	Description
loading	boolean	true	True while enrichment is in progress.
data	EnrichedProfile null	null	The complete enriched profile once available.
error	string null	null	Error message when enrichment fails.
syncing	boolean	false	Whether a manual sync is running.
resolvedRef	React.useRef<boolean>	false	Prevents duplicate resolution after completion/failure.
hasTriggeredSyncRef	React.useRef<boolean>	false	Prevents duplicate sync calls on re-renders.
wsRef	React.useRef<WebSocket null>	null	Holds the active WebSocket connection.
reconnectTimeoutRef	React.useRef<Timeout null>	null	Holds the pending reconnection timer for cleanup.

Key event handling:

Event	Handler	Logic
-------	---------	-------

Page load / mount	useEffect candidateUuid	on	Extracts candidate_uuid from the URL, stores it in context, triggers sync if not already running.
WebSocket connect	connectWebSocket()		Opens the analysis WebSocket, closing any existing socket first, and registers the on* handlers.
WebSocket message	ws.onmessage		On ENRICHED: stores data, marks resolved, closes socket. On FAILED: sets error, marks resolved, closes socket.
WebSocket unexpected close	ws.onclose		Ignores manual/resolved closes; otherwise polls HTTP status and reconnects after a 2.5s backoff.
Accordion header click	toggle("github" "linkedin")		Single-open accordion behaviour for the GitHub and LinkedIn cards.
Career timeline header click	setExpanded()		Expands/collapses the career timeline, independent of the accordion.
"Show Delta" toggle	setShowBoth()		Toggles the radar chart between post-only and pre-/post-enrichment comparison.
Automatic sync on load	triggerSync() in useEffect		Guards against duplicate calls, calls syncCandidateProfile(), updates the syncing indicator.
Component unmount	cleanup in useEffect		Closes the WebSocket gracefully, marks it manually closed, clears the reconnect timer.

Performance considerations:

- Exponential backoff (2.5s) on WebSocket reconnection prevents connection spam.
- Sync/connection flags live in refs, not state, to avoid re-renders.
- A WeakSet tracks manually closed sockets to prevent memory leaks.
- Components render conditionally, once data is available.

Accessibility features:

- Semantic HTML with a proper heading hierarchy.
- Accordion cards are keyboard-accessible via Tab and Enter.
- Loading/error states use ARIA labels for screen readers.
- All text meets WCAG AA colour-contrast requirements.

5.2.6. *Screen 6 – Interview Scheduling View*

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

The Interview Scheduling screen provides a three-panel layout for managing interview scheduling with Google Calendar integration.

Screen layout:

- Left panel — Interviewer list: Shows all available interviewers with name, role, initials (colour-coded), and calendar connection status. Users can select one or more interviewers for availability lookup.
- Centre panel — Available slots timeline: Displays the computed free slots for selected interviewers within a chosen date range. Each slot shows the interviewer name, date, and time. Clicking a slot highlights it for confirmation.
- Right panel — Weekly calendar view: A visual calendar showing the current week with confirmed slots displayed as coloured blocks. Confirmed interviews include the candidate name and interviewer details.

State management:

- Available slots are fetched from the backend on each availability check request; previous results are cleared when a new search is initiated.
- Confirmed slots are persisted to the backend and stored in `confirmed_slots.json` for future reference.
- Loading states for FreeBusy API calls and confirmation requests are tracked via separate boolean flags to prevent duplicate submissions.

Key event handling:

- Check Availability — Validates that at least one interviewer is selected and date range is valid. Sends GET `/api/scheduling/availability` with interviewer IDs, `dateFrom`, and `dateTo`. Displays results in the centre panel.
- Select Slot — Highlights the selected time slot and enables the confirmation button.
- Confirm Interview — Sends POST `/api/scheduling/confirm` with candidate UUID, interviewer ID, and selected time range. On success, updates the confirmed slots list and shows a success notification.

Role-based access:

This screen is only accessible to users with hr or admin role. Attempting to access /schedule as tech_lead redirects to the candidate profile view with a permissions error message.

6 AI Usage Declaration

In alignment with academic integrity guidelines and responsible engineering principles, our team utilized Generative AI assistance (specifically Google Gemini) strictly as an **augmented productivity tool** throughout the Design and Implementation phases of the SmartATS project.

The boundaries and specific applications of AI usage within this project are defined as follows:

- **Architectural Brainstorming & Formatting:** AI was leveraged to assist in structuring initial documentation layouts, refining English grammar, and formatting complex technical specifications (such as LaTeX mathematical formulas, markdown tables, and sequence lists) to ensure professional academic standards.
- **Diagram & Syntax Verification:** Generative AI served as a peer-review partner to validate the syntax of Mermaid.js diagrams and verify physical database constraint notations against standardized Extended Entity-Relationship (EER) modeling rules.
- **Boilerplate Code Suggestion:** During the implementation phase, AI was used to generate low-level, repetitive code scaffolding (such as standard Pydantic models, FastAPI routing boilerplate, and basic React component layouts).

Human Oversight & Accountability Statement:

Zero Automated Decision-Making: All core architectural decisions—including the system boundary design, Multi-Agent orchestration logic, ABAC security rules, and algorithm selections (such as the Sweep-Line scheduler)—were conceived, debated, and finalized strictly by team members. Every piece of AI-suggested code and documentation artifact underwent manual code review, refactoring, and integration testing before being

merged into the master project repository. The team assumes 100% full accountability for all deliverables, source code, and technical documentation presented in this report.

7 Presentation

Link to video: <https://youtu.be/-S9gvktXv3E>

Meeting Overview:

To ensure seamless integration and strict adherence to project deadlines, the engineering team conducted a structured design synchronization meeting. This session focused on validating the **Software Design Document (SDD)** artifacts, resolving architectural constraints across modules, and verifying individual task progress within Jira.

- **Session Objective:** Review EER conceptual schemas, Class Diagrams, ABAC security policies, and UI event specifications.
- **Key Outcome:** 100% task alignment across all 4 sub-systems with equal 25% workload contribution validated and merged for final delivery.

8 Reflective Report

This section provides a critical evaluation of Template2-Design.docx, identifying which sections directly facilitated our system implementation and which proved redundant, supported by concrete examples from our development workflow.

8.1 Most Helpful Sections for Implementation

- **Section 3.2 (Class Diagram Specifications):** This section served as the exact coding blueprint for backend development. Having to explicitly declare class attributes, method signatures, access modifiers (public, private), and input/output types reduced implementation ambiguity to zero.
 - *Example:* Specifying the ABACPolicyEngine class with method signature `mask_pii_payload(payload: dict, role: RoleEnum) -> dict` allowed our backend engineer to implement the exact FastApi middleware logic in minutes without re-architecting data structures.
- **Section 4 (Physical Database Specifications):** The data specification matrix directly accelerated our database migration scripts.
 - *Example:* Mapping the conceptual Candidate entity into physical PostgreSQL column types—specifically specifying VECTOR(1536) for pgvector embeddings and JSONB for unstructured GitHub repository metadata—prevented schema mismatches and allowed us to write precise SQLAlchemy models right away.
- **Section 5 (UI Event Handling Specification):** Mapping UI component states to backend API triggers bridged the gap between React state management and asynchronous APIs.
 - *Example:* Documenting the onDrop event for *Screen 1 (Workspace)* specifying an immediate HTTP 202 Accepted response alongside a

WebSocket handshake URL (/ws/v1/ingestion/{job_id}) enabled parallel frontend-backend integration without waiting for service deployment.

8.2 Unnecessary or Redundant Sections

- **Section 2 vs Section 4 Redundancy (Overlapping Data Models):** Defining detailed textual entity descriptions in Section 2 (Conceptual Model) felt largely redundant once Section 4 (Physical Database Specification) was completed.
 - *Example:* Explaining candidate fields (e.g., phone_number, salary_expectation) in prose within Section 2 added little value compared to the concrete data specification table in Section 4, which already defined column constraints, nullability, and primary/foreign keys needed for implementation.
- **Static UI Layout Diagrams in Document:** Static screen mockups in Section 5 became quickly outdated as UI components evolved during development. Interactive prototypes (such as Figma links or live storybooks) proved significantly more effective for implementation than static image inserts.

8.3 Conclusion

While high-level textual descriptions can be streamlined to reduce documentation overhead, the concrete **Class Specifications**, **Physical Schema Tables**, and **UI Event Matrices** are indispensable. They transformed our team's workflow from ad-hoc coding into disciplined, architecture-first execution.